

计算机科学与技术学院

系统开发工具基础

第二周实验报告

学生姓名： 杜铭昊

学号： 25020007021

授课教师： 周小伟

实验环境： Ubuntu Linux 虚拟机

报告内容： 课堂第 5-8 题与 10 个拓展实例

实验日期： 2026 年 8 月 31 日

目录

1 实验说明与环境	3
2 第 5 题：控制一个可清理的后台任务	3
2.1 实验原理	3
2.2 步骤 1：创建并检查 worker 脚本	3
2.3 步骤 2：挂起、恢复并检查作业状态	4
2.4 步骤 3：自动取得 PID 并正常终止	5
2.5 实验结果与错误反思	6
3 第 6 题：语义重构与本地开发反馈	7
3.1 实验原理	7
3.2 步骤 1：建立环境和基线文件	7
3.3 步骤 2：跳转到定义与查找引用	8
3.4 步骤 3：使用 Rename Symbol 完成语义重构	8
3.5 步骤 4：ruff 发现并修复未使用导入	8
3.6 步骤 5：最终测试与运行结果	9
3.7 实验结果与错误反思	10
4 第 7 题：用调试器定位归并排序缺陷	11
4.1 实验原理	11
4.2 步骤 1：运行缺陷版本并保存错误现象	11
4.3 步骤 2：在 pdb 中设置断点并观察变量	11
4.4 步骤 3：完成最小修复	12
4.5 步骤 4：增加两项 pytest 测试并验收	13
4.6 实验结果与错误反思	14
5 第 8 题：先测量，再优化慢速词频程序	15
5.1 实验原理	15
5.2 步骤 1：生成固定输入	15
5.3 步骤 2：原程序两次计时并进行 cProfile 分析	15
5.4 步骤 3：使用集合优化且保持输出不变	16
5.5 步骤 4：优化后复测并计算加速比	17
5.6 实验结果与错误反思	18
6 10 个相关拓展实例	19
6.1 实例 1：后台监控任务的 TERM 清理	19
6.2 实例 2：CPU 任务的后台运行与等待	20
6.3 实例 3：生产者管道与信号结束	21
6.4 实例 4：pytest 驱动的计算器验证	22
6.5 实例 5：ruff 发现并修复代码质量问题	23

6.6 实例 6: 失败测试指导用户名函数修复	24
6.7 实例 7: pdb 定位二分查找边界错误	25
6.8 实例 8: 递归斐波那契性能分析与优化	26
6.9 实例 9: 列表与集合成员查询基准	27
6.10 实例 10: 整体读取与流式读取的内存比较	28
7 Git 提交与 GitHub 记录	29
8 综合结论与错误反思	32
A 最终验收清单	32

1 实验说明与环境

本实验严格对应第二周课堂第 5-8 题，在 Ubuntu 虚拟机中完成真实运行，并保存代码、标准输出、标准错误、测试结果、性能数据和桌面截图。课堂题按照题目步骤逐项说明；10 个拓展实例与课堂内容相关，采用较简洁的“原理-过程-结果-反思”结构。仓库地址为<https://github.com/ddd666j/system-tools-week2>，实验按阶段提交，拓展实例每两个形成一次独立 commit。

项目	配置与验证方式
操作系统	Ubuntu Linux 虚拟机；Windows 用于同步材料与 GitHub 推送
主要工具	Bash 任务控制、Python 3.12、pylsp、ruff、pytest、pdb、cProfile、Git、XeLaTeX
目录结构	q05--q08 保存课堂题；practice/instance01--10 保存拓展实例；report 保存报告
证据	每一步保存命令、输出、结果文件和 Ubuntu 桌面截图；Git 提交记录另行保存

2 第 5 题：控制一个可清理的后台任务

2.1 实验原理

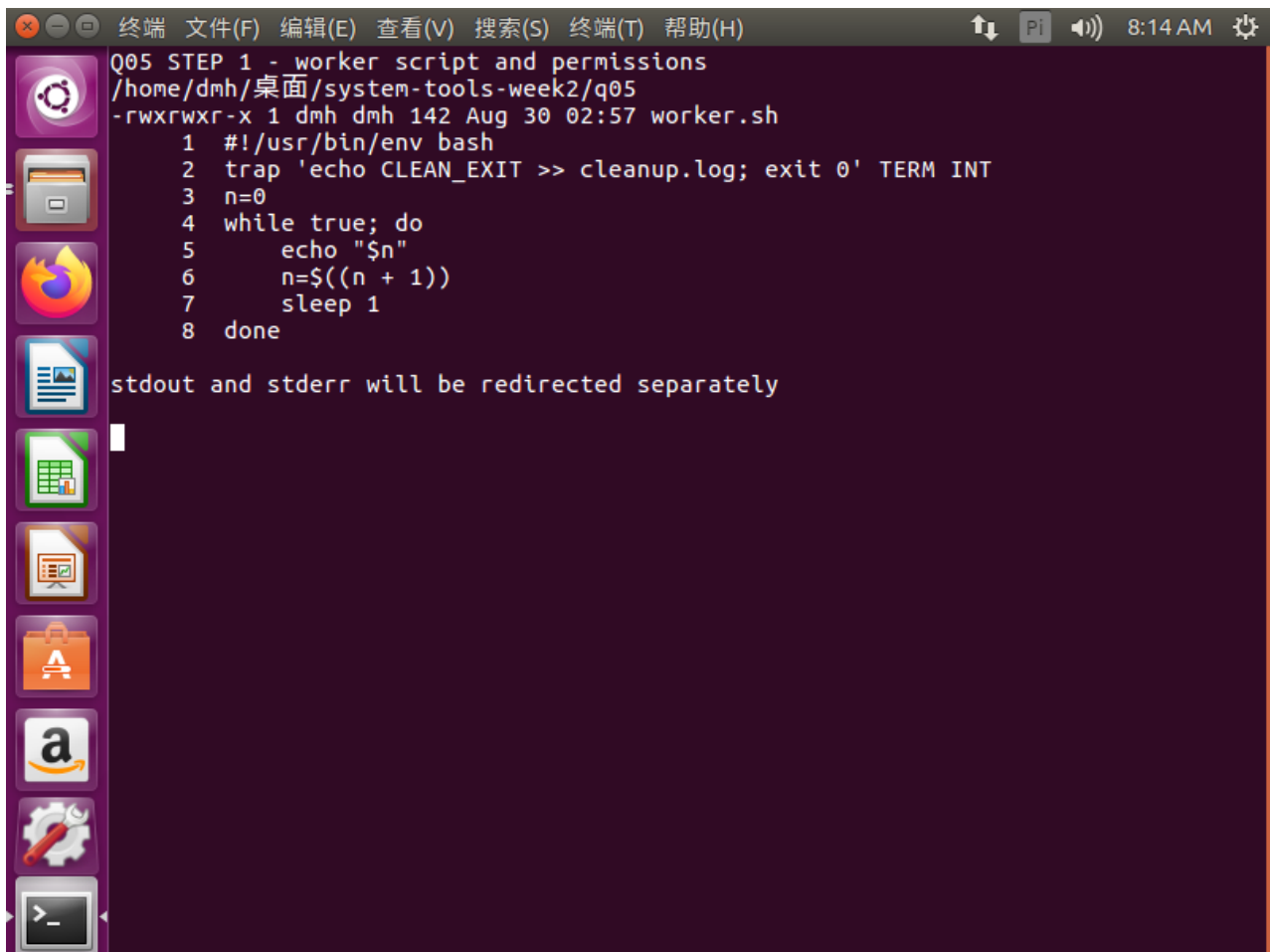
Shell 任务控制以作业为单位管理前台和后台进程。Ctrl-Z 由终端向前台进程组发送 SIGTSTP，使任务暂停；bg 向暂停任务发送 SIGCONT 并让其在后台继续；jobs 显示当前 Shell 维护的作业表。jobs -p 能直接取得作业 PID，避免人工抄写出错。脚本用 trap 捕获 SIGTERM 或 SIGINT，在退出前写入清理标志。SIGKILL 无法被捕获，因此题目禁止 kill -9。

2.2 步骤 1：创建并检查 worker 脚本

在 q05 目录创建 worker.sh，循环每秒输出计数值，并为 TERM 和 INT 注册清理处理函数。赋予执行权限后检查文件内容和权限。

```
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done

chmod +x worker.sh
./worker.sh >stdout.log 2>stderr.log
```



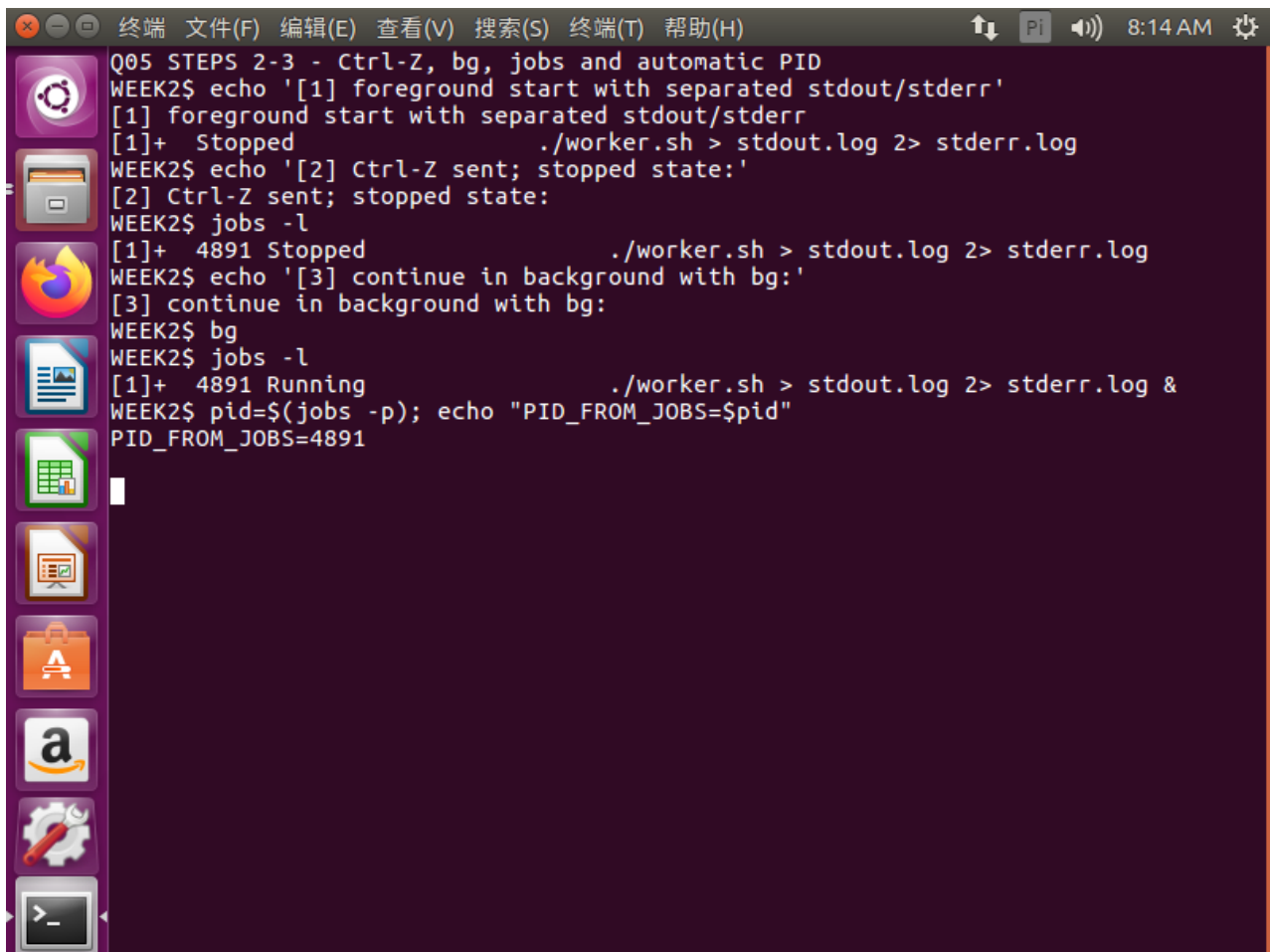
```
终端 文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H) 8:14 AM
Q05 STEP 1 - worker script and permissions
/home/dmh/桌面/system-tools-week2/q05
-rwxrwxr-x 1 dmh dmh 142 Aug 30 02:57 worker.sh
1 #!/usr/bin/env bash
2 trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
3 n=0
4 while true; do
5     echo "$n"
6     n=$((n + 1))
7     sleep 1
8 done
stdout and stderr will be redirected separately
```

图 1: 第 5 题步骤 1: 脚本、权限以及前台启动与输出重定向

2.3 步骤 2: 挂起、恢复并检查作业状态

脚本前台运行后按 `Ctrl-Z`。终端报告 `Stopped`，说明前台进程组已暂停。执行 `jobs` 复核状态，再执行 `bg` 使它在后台继续。再次执行 `jobs` 可看到 `Running`。由于 `stdout` 已经重定向，后台任务不会持续污染终端，但 `stdout.log` 中的计数继续增长。

```
# 在前台运行期间按 Ctrl-Z
jobs
bg
jobs
wc -l stdout.log stderr.log
```



The screenshot shows a terminal window with a menu bar (终端, 文件(F), 编辑(E), 查看(V), 搜索(S), 终端(T), 帮助(H)) and a status bar (8:14 AM). The terminal output is as follows:

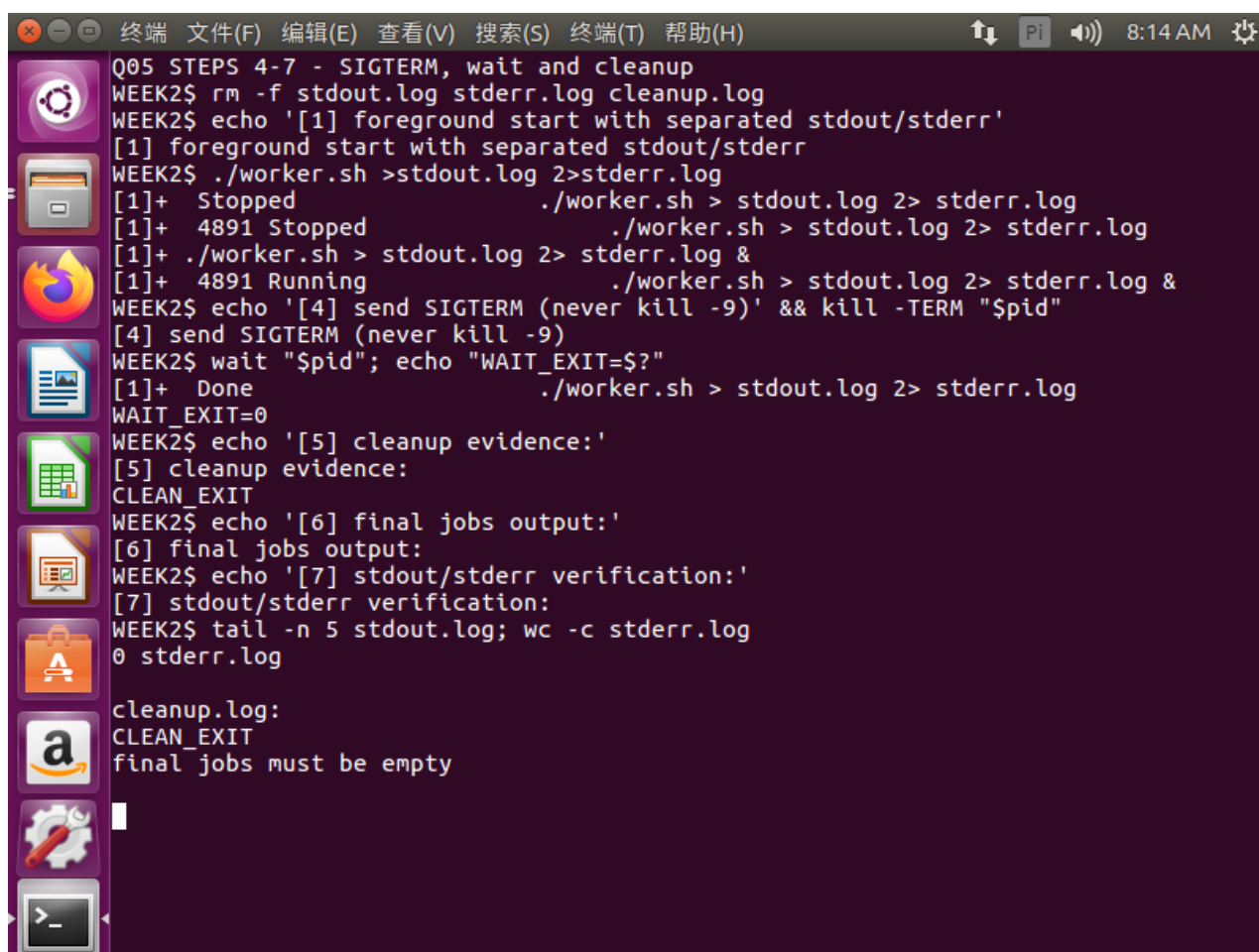
```
Q05 STEPS 2-3 - Ctrl-Z, bg, jobs and automatic PID
WEEK2$ echo '[1] foreground start with separated stdout/stderr'
[1] foreground start with separated stdout/stderr
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
WEEK2$ echo '[2] Ctrl-Z sent; stopped state:'
[2] Ctrl-Z sent; stopped state:
WEEK2$ jobs -l
[1]+  4891 Stopped              ./worker.sh > stdout.log 2> stderr.log
WEEK2$ echo '[3] continue in background with bg:'
[3] continue in background with bg:
WEEK2$ bg
WEEK2$ jobs -l
[1]+  4891 Running               ./worker.sh > stdout.log 2> stderr.log &
WEEK2$ pid=$(jobs -p); echo "PID_FROM_JOBS=$pid"
PID_FROM_JOBS=4891
```

图 2: 第 5 题步骤 2: Ctrl-Z 挂起、bg 恢复和 jobs 状态验证

2.4 步骤 3: 自动取得 PID 并正常终止

用命令替换接收 `jobs -p` 输出，不手工填写 PID。向任务发送 `SIGTERM`，随后 `wait` 等待子进程彻底结束并收集退出状态。最后检查清理日志、作业表及输出文件。

```
pid=$(jobs -p)
printf 'PID=%s\n' "$pid"
kill -TERM "$pid"
wait "$pid"
printf 'wait_status=%s\n' "$?"
cat cleanup.log
jobs
wc -l stdout.log stderr.log
```



```
Q05 STEPS 4-7 - SIGTERM, wait and cleanup
WEEK2$ rm -f stdout.log stderr.log cleanup.log
WEEK2$ echo '[1] foreground start with separated stdout/stderr'
[1] foreground start with separated stdout/stderr
WEEK2$ ./worker.sh >stdout.log 2>stderr.log
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
[1]+ 4891 Stopped              ./worker.sh > stdout.log 2> stderr.log
[1]+ ./worker.sh > stdout.log 2> stderr.log &
[1]+ 4891 Running              ./worker.sh > stdout.log 2> stderr.log &
WEEK2$ echo '[4] send SIGTERM (never kill -9)' && kill -TERM "$pid"
[4] send SIGTERM (never kill -9)
WEEK2$ wait "$pid"; echo "WAIT_EXIT=$?"
[1]+  Done                      ./worker.sh > stdout.log 2> stderr.log
WAIT_EXIT=0
WEEK2$ echo '[5] cleanup evidence:'
[5] cleanup evidence:
CLEAN_EXIT
WEEK2$ echo '[6] final jobs output:'
[6] final jobs output:
WEEK2$ echo '[7] stdout/stderr verification:'
[7] stdout/stderr verification:
WEEK2$ tail -n 5 stdout.log; wc -c stderr.log
0 stderr.log

cleanup.log:
CLEAN_EXIT
final jobs must be empty
```

图 3: 第 5 题步骤 3: SIGTERM、wait、CLEAN_EXIT 与任务退出验证

2.5 实验结果与错误反思

stdout 和 stderr 分别写入 stdout.log 与 stderr.log; 任务经历 Stopped 和 Running 状态;PID 由 jobs -p 自动取得;SIGTERM 后 wait 返回 0;cleanup.log 出现 CLEAN_EXIT;最终 jobs 为空, stderr.log 为 0 字节。

反思: 不能用 kill -9 代替 SIGTERM, 因为 SIGKILL 不会触发 trap, 清理文件将无法生成。还应在同一个交互式 Shell 中执行任务控制命令, 因为作业表属于当前 Shell。

3 第 6 题：语义重构与本地开发反馈

3.1 实验原理

语言服务器协议 (LSP) 让编辑器通过程序语法与符号关系实现跳转到定义、查找引用和重命名。语义重命名不同于文本替换：服务器返回 `WorkspaceEdit`，只修改对应符号，并能同步处理定义、导入和测试引用。ruff 负责静态检查和自动修复，pytest 负责行为验证；二者共同形成快速本地反馈环。

3.2 步骤 1：建立环境和基线文件

创建 `math_utils.py`、`app.py` 和 `test_math_utils.py`，先检查 Python、pylsp、ruff 与 pytest 版本，再运行原始程序与测试，确认重构前基线正确。

```
python3 --version
pylsp --version
ruff --version
pytest --version
python3 app.py
pytest -q
```

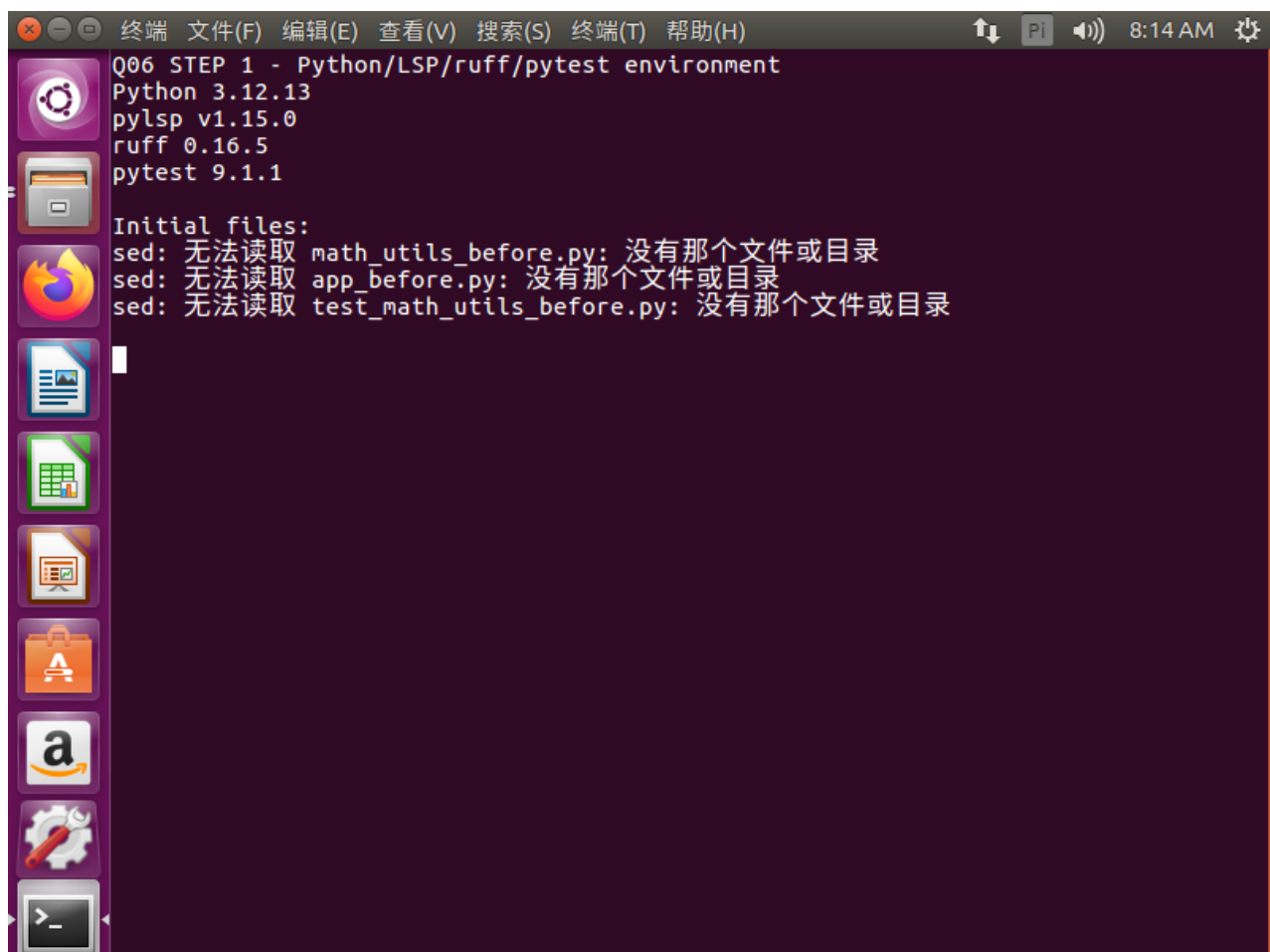


图 4：第 6 题步骤 1：开发工具环境与初始代码验证

3.3 步骤 2：跳转到定义与查找引用

启用 Python 语言服务器,定位 `app.py` 中的 `total_price` 调用。跳转到定义返回 `math_utils.py` 中的函数定义；查找引用得到定义、应用程序导入与调用、测试导入与调用共 5 个位置。这一步确认语言服务器理解的是符号关系，而不是简单字符串搜索。

3.4 步骤 3：使用 Rename Symbol 完成语义重构

在函数符号处请求重命名为 `calculate_total`。语言服务器返回跨 3 个文档的编辑，应用全部 `WorkspaceEdit` 后，定义、`app.py` 和测试文件中的引用同步更新。

```
# math_utils.py
def calculate_total(price: float, count: int) -> float:
    return price * count

# app.py
from math_utils import calculate_total
print(calculate_total(12.5, 4))
```

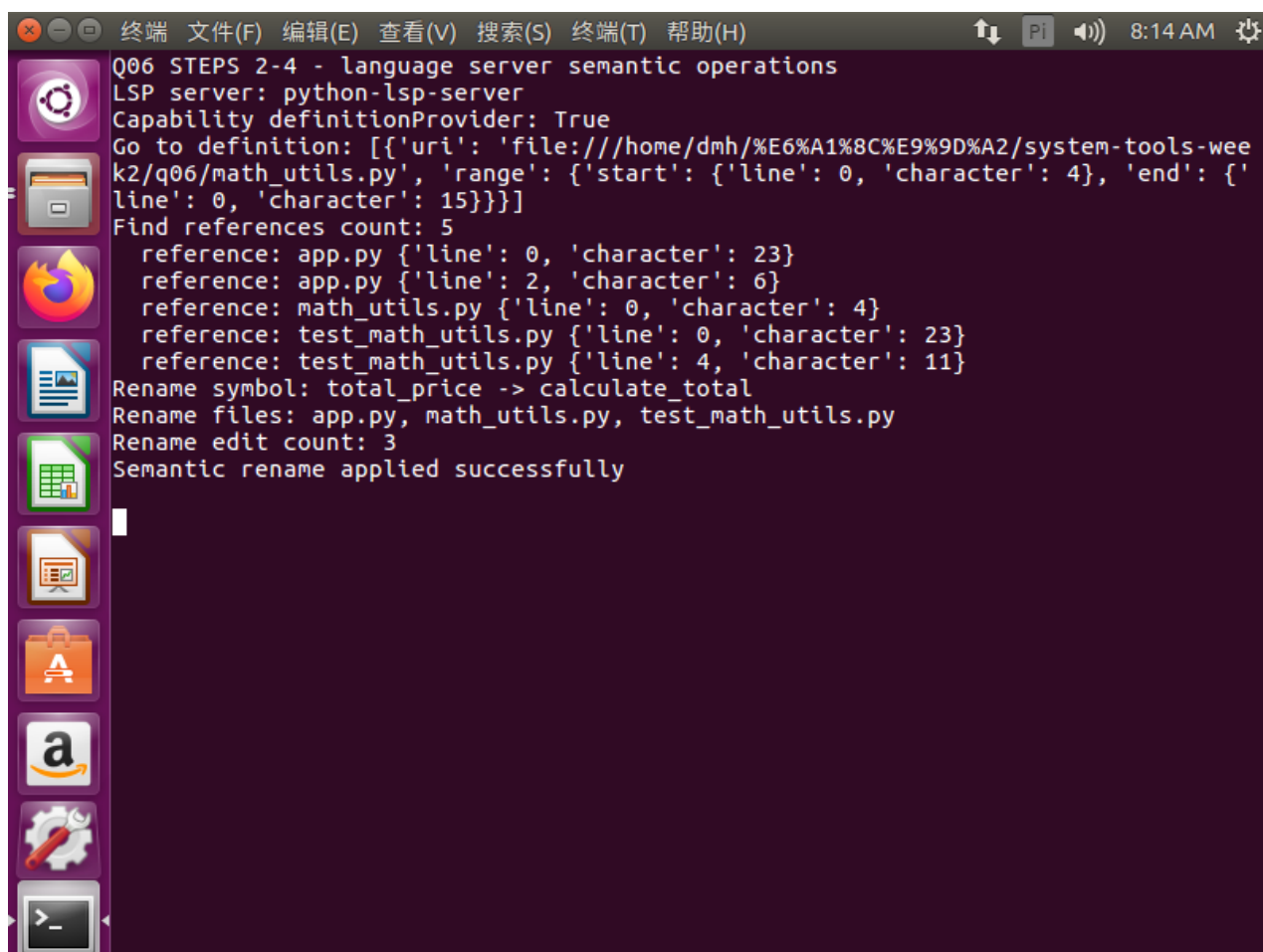


图 5：第 6 题步骤 2-3：定义跳转、5 处引用和跨文件语义重命名

3.5 步骤 4：ruff 发现并修复未使用导入

按题意在 `app.py` 临时加入 `import os`。ruff 报告 F401 未使用导入，说明静态检查能够在运行前发现无效依赖。执行自动修复后，`import os` 被删除，再次检查通过。

```
ruff check .  
ruff check . --fix  
ruff check .
```

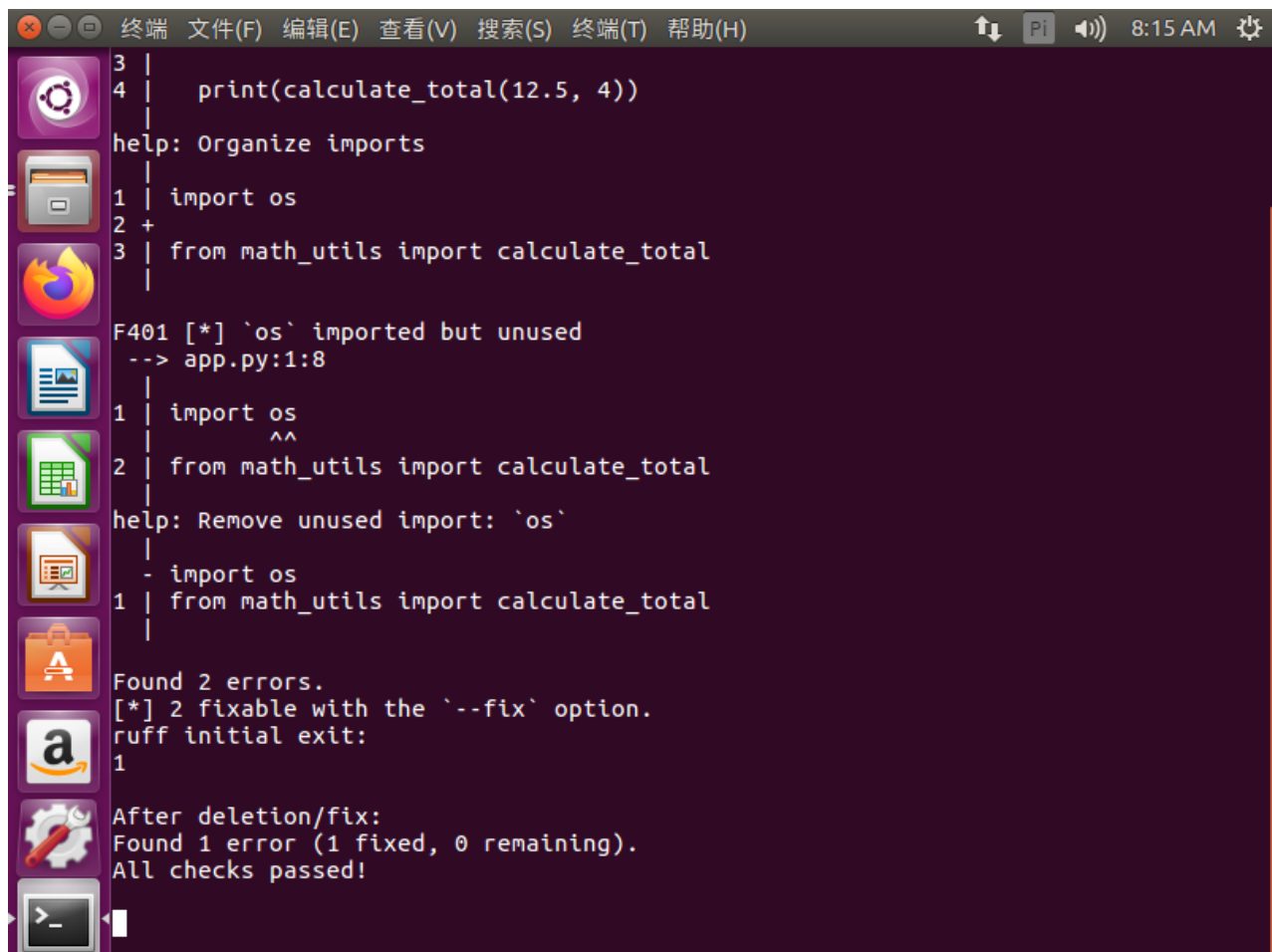
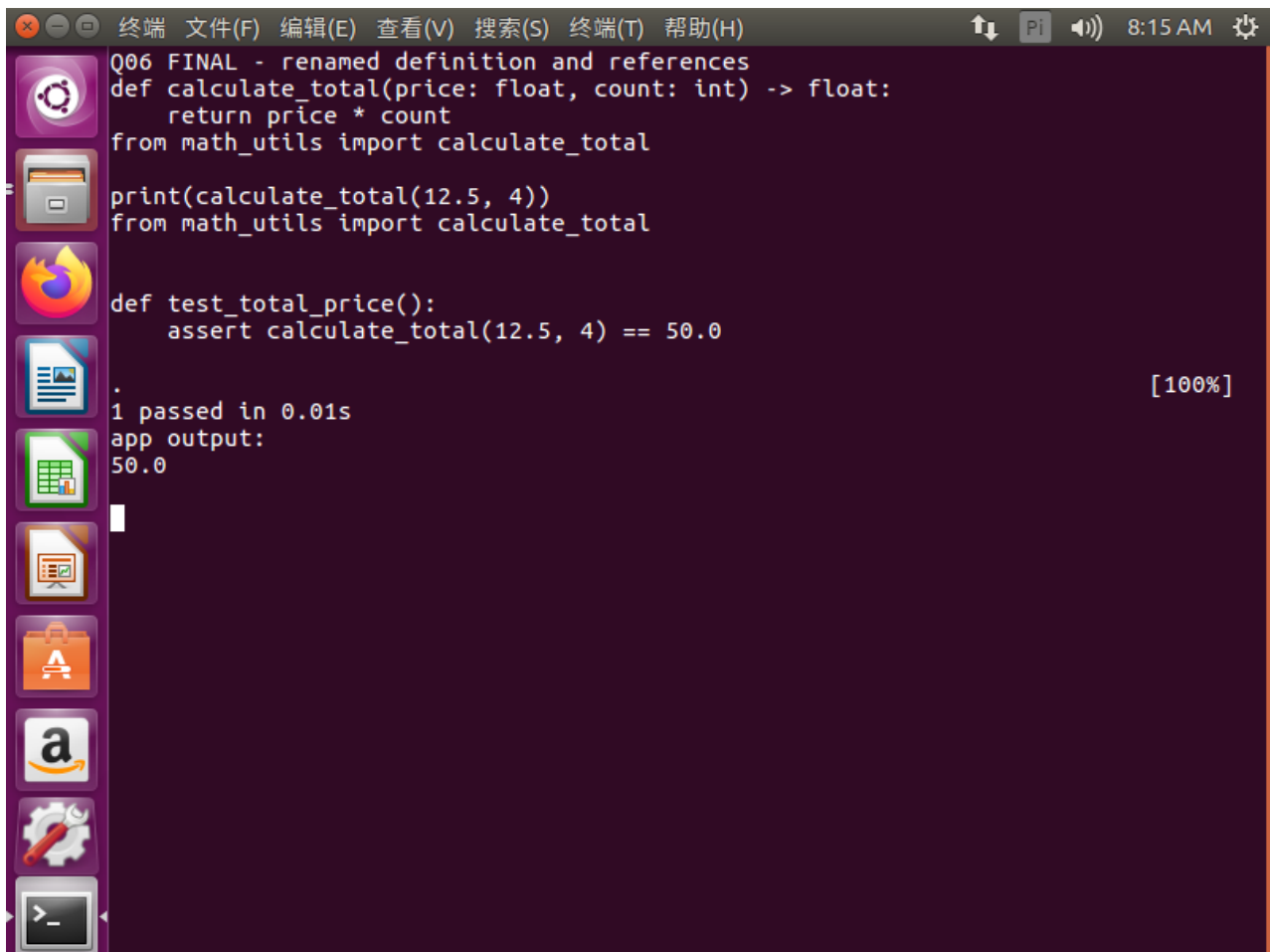


图 6: 第 6 题步骤 4: ruff 报告 F401 并完成自动修复

3.6 步骤 5: 最终测试与运行结果

依次运行 ruff、pytest 和应用程序。ruff 无错误，pytest 显示 1 passed，应用输出 50.0，证明代码风格、测试行为和实际运行均正确。

A screenshot of a terminal window with a dark purple background. The window title bar shows standard Linux window controls and a menu bar with options: 终端 (Terminal), 文件(F) (File), 编辑(E) (Edit), 查看(V) (View), 搜索(S) (Search), 终端(T) (Terminal), and 帮助(H) (Help). The status bar on the right shows a Pi icon, a speaker icon, and the time 8:15 AM. The terminal content shows Python code for a function `calculate_total` and its test `test_total_price`. The test passes, and the application output is `50.0`. The left sidebar of the terminal window displays various application icons including a gear, a folder, a Firefox browser, a document, a spreadsheet, a presentation, a shopping bag, an Amazon logo, a settings gear, and a terminal icon.

```
Q06 FINAL - renamed definition and references
def calculate_total(price: float, count: int) -> float:
    return price * count
from math_utils import calculate_total

print(calculate_total(12.5, 4))
from math_utils import calculate_total

def test_total_price():
    assert calculate_total(12.5, 4) == 50.0

.
1 passed in 0.01s
app output:
50.0
```

图 7: 第 6 题步骤 5: ruff、pytest 与应用输出的最终验收

3.7 实验结果与错误反思

pylsp 完成定义跳转并找到 5 个定义/引用位置; Rename Symbol 产生 3 个文档编辑; ruff 成功发现并删除未使用的 `import os`; pytest 结果为 1 passed; 应用输出 50.0。

反思: 不能仅用全局字符串替换冒充语义重构, 它可能误改注释、同名局部变量或字符串。实现 LSP 客户端时还要同时处理 `documentChanges` 和 `changes` 两种 `WorkspaceEdit` 形式, 并确保全部编辑均被应用。

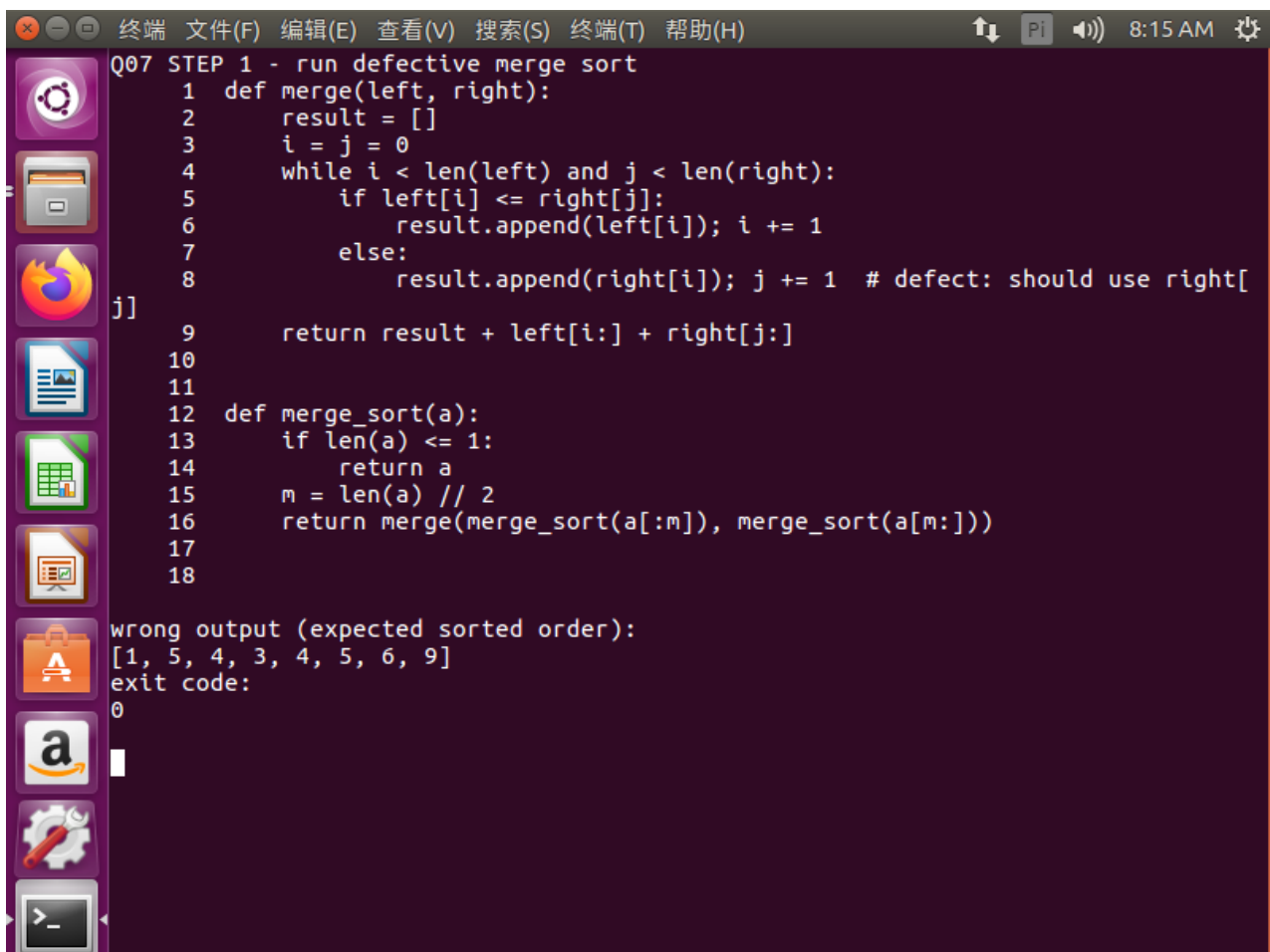
4 第 7 题：用调试器定位归并排序缺陷

4.1 实验原理

归并排序递归拆分输入，再由 `merge` 函数合并两个已经有序的子序列。合并时 `i` 索引左序列，`j` 索引右序列；若右侧元素更小，应追加 `right[j]`。调试器通过断点和变量观察把“结果错误”缩小到具体状态与语句，最小修复可以减少引入新问题的风险。

4.2 步骤 1：运行缺陷版本并保存错误现象

按照题目代码运行给定输入。程序没有抛出异常、退出码也是 0，但输出为 `[1, 5, 4, 3, 4, 5, 6, 9]`，不满足非降序排列，且两个 1 没有正确保留。说明仅检查退出码不足以判断算法正确性。



```
Q07 STEP 1 - run defective merge sort
1 def merge(left, right):
2     result = []
3     i = j = 0
4     while i < len(left) and j < len(right):
5         if left[i] <= right[j]:
6             result.append(left[i]); i += 1
7         else:
8             result.append(right[i]); j += 1 # defect: should use right[j]
9     return result + left[i:] + right[j:]
10
11
12 def merge_sort(a):
13     if len(a) <= 1:
14         return a
15     m = len(a) // 2
16     return merge(merge_sort(a[:m]), merge_sort(a[m:]))
17
18
wrong output (expected sorted order):
[1, 5, 4, 3, 4, 5, 6, 9]
exit code:
0
```

图 8：第 7 题步骤 1：缺陷版本运行成功但排序结果错误

4.3 步骤 2：在 pdb 中设置断点并观察变量

进入 `pdb`，在 `merge` 函数错误的追加语句处设置断点，运行到右分支后检查 `i`、`j`、`left[i]` 与 `right[j]`。一次关键状态为 `i=0`，`j=0`，`left[i]=3`，`right[j]=1`，按算法应追加右侧当前元素 `right[j]`，但代码错误读取 `right[i]`。

```
python3 -m pdb merge_sort_buggy.py
(Pdb) break merge_sort_buggy.py:<append所在行>
(Pdb) continue
(Pdb) p i
```

```
(Pdb) p j
(Pdb) p left[i]
(Pdb) p right[j]
```

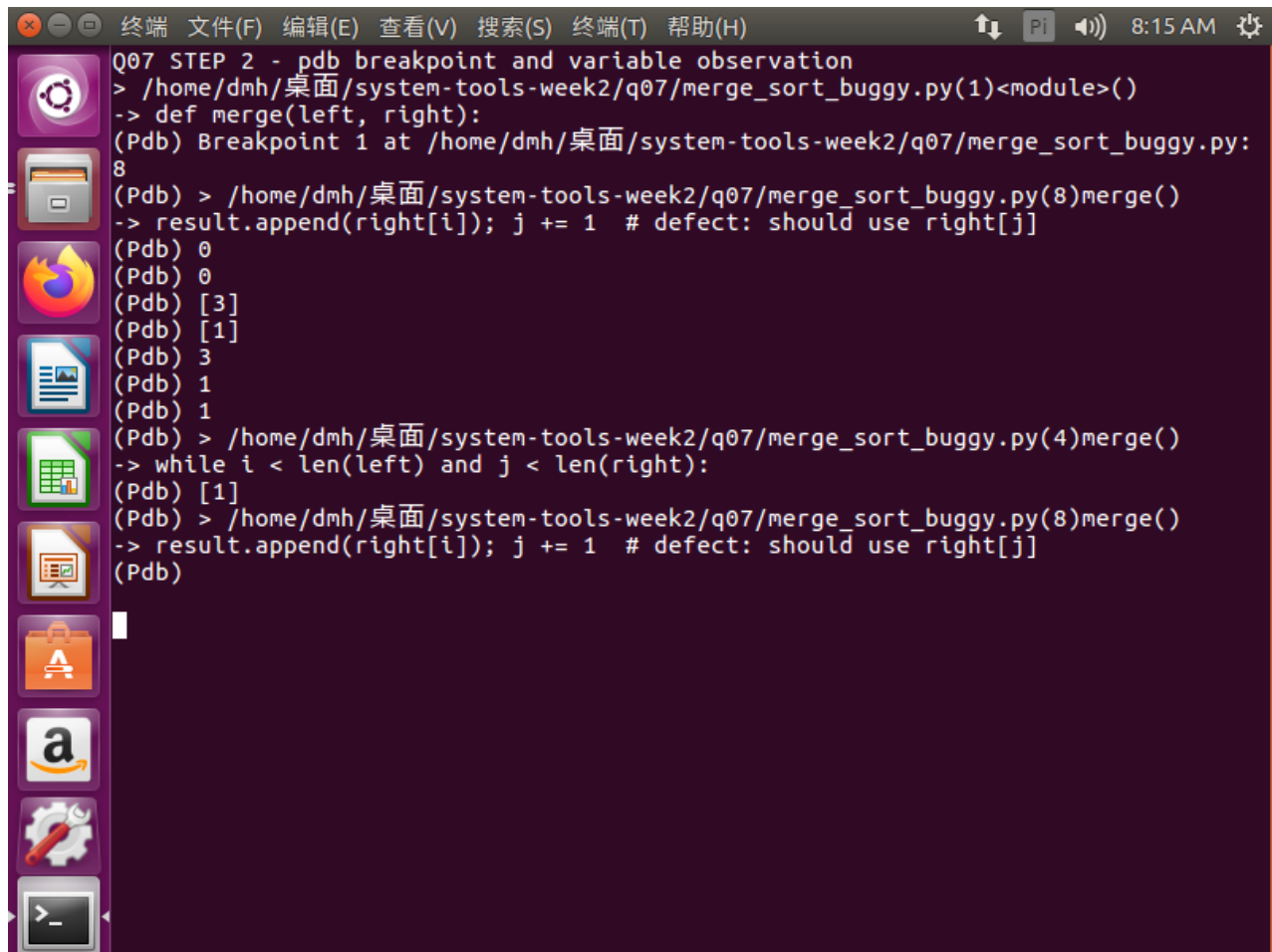
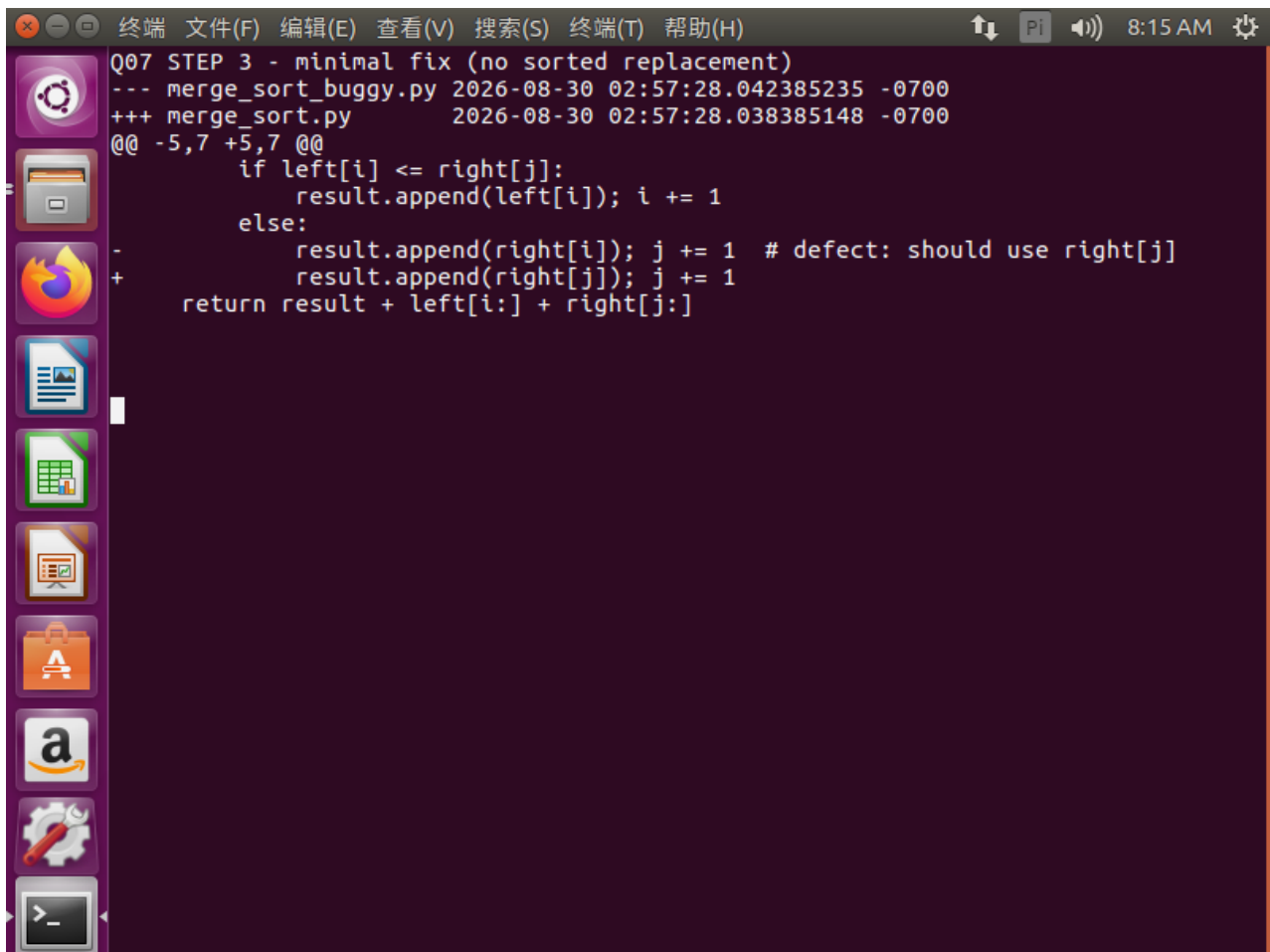


图 9：第 7 题步骤 2：断点处观察 i、j 及左右当前元素

4.4 步骤 3：完成最小修复

只把右分支的 `result.append(right[i])` 改成 `result.append(right[j])`，其余归并排序结构保持不变，也没有使用 `sorted` 替代算法。

```
else:
    result.append(right[j])
    j += 1
```



```
Q07 STEP 3 - minimal fix (no sorted replacement)
--- merge_sort_buggy.py 2026-08-30 02:57:28.042385235 -0700
+++ merge_sort.py       2026-08-30 02:57:28.038385148 -0700
@@ -5,7 +5,7 @@
     if left[i] <= right[j]:
         result.append(left[i]); i += 1
     else:
-        result.append(right[i]); j += 1 # defect: should use right[j]
+        result.append(right[j]); j += 1
     return result + left[i:] + right[j:]
```

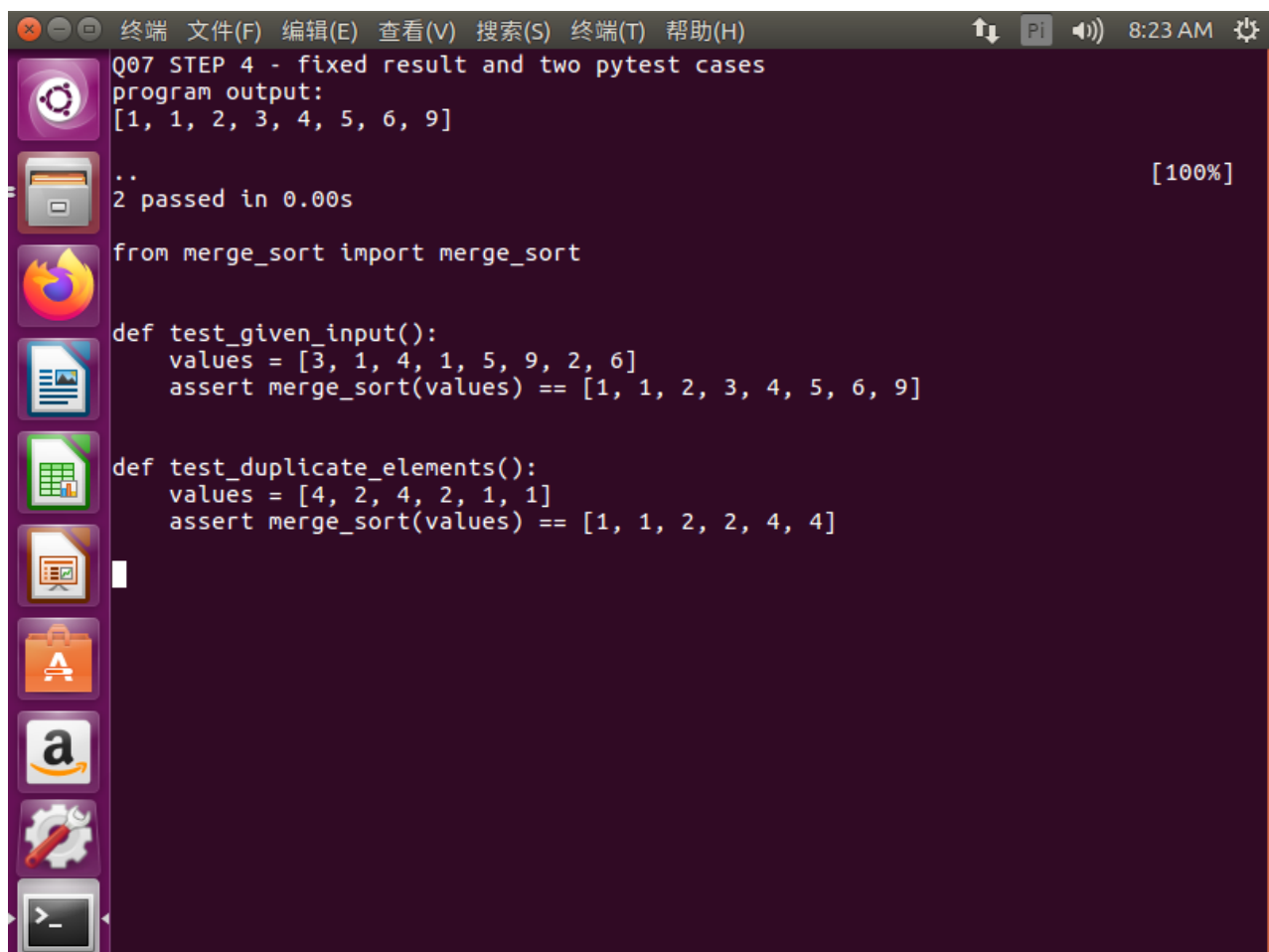
图 10: 第 7 题步骤 3: 仅修正错误索引的最小补丁

4.5 步骤 4: 增加两项 pytest 测试并验收

测试一覆盖题目给定输入, 期望结果为 [1,1,2,3,4,5,6,9]; 测试二使用含重复元素的列表, 验证重复值不会丢失。执行 pytest 后两项均通过, 直接运行修复版也得到正确结果。

```
def test_given_input():
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]

def test_duplicates():
    assert merge_sort([2, 2, 1, 3, 1]) == [1, 1, 2, 2, 3]
```



```
Q07 STEP 4 - fixed result and two pytest cases
program output:
[1, 1, 2, 3, 4, 5, 6, 9]

..                                     [100%]
2 passed in 0.00s

from merge_sort import merge_sort

def test_given_input():
    values = [3, 1, 4, 1, 5, 9, 2, 6]
    assert merge_sort(values) == [1, 1, 2, 3, 4, 5, 6, 9]

def test_duplicate_elements():
    values = [4, 2, 4, 2, 1, 1]
    assert merge_sort(values) == [1, 1, 2, 2, 4, 4]
```

图 11: 第 7 题步骤 4: 给定输入与重复元素测试全部通过

4.6 实验结果与错误反思

缺陷版本输出无序列表; pdb 在错误分支捕获到关键索引和元素值; 最小修复仅将 `right[i]` 改为 `right[j]`; 修复后给定结果正确, 两项 pytest 测试全部通过。

反思: 最初若把断点设在 `else` 关键字所在行, pdb 不会按预期暂停, 因为关键字本身不是独立可执行语句。应把断点设置在实际的 `append` 语句上。测试不仅应覆盖给定样例, 还应验证重复元素, 防止排序正确但元素个数错误。

5 第 8 题：先测量，再优化慢速词频程序

5.1 实验原理

原程序用列表保存不同词，每次 `word not in unique` 都可能线性扫描列表。若总词数为 n 、不同词数为 u ，总体复杂度近似 $O(nu)$ 。集合基于哈希表，平均成员查询接近 $O(1)$ ，因此整体可接近 $O(n)$ 。性能优化必须遵循“先测量、再定位、后修改、再对比”，并用相同输入与一致输出验证优化没有改变语义。

5.2 步骤 1：生成固定输入

使用固定随机种子 20260831，从 1000 个候选词中生成 30000 个词并保存为 `words.txt`。固定种子确保多次实验读取相同数据，便于复现和公平比较；程序不能把最终不同词数量写死为 1000。

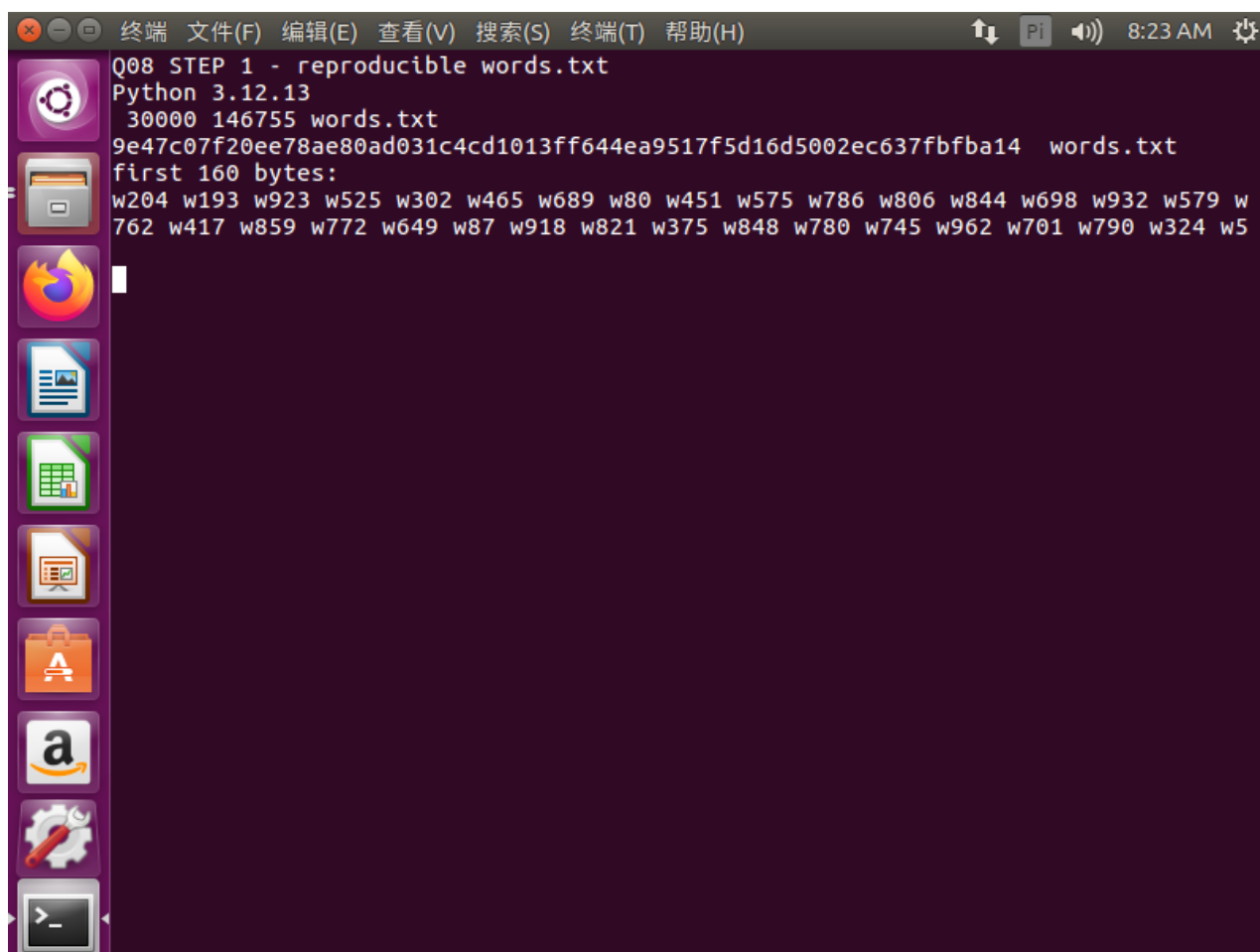


图 12：第 8 题步骤 1：固定随机种子生成 30000 词输入

5.3 步骤 2：原程序两次计时并进行 cProfile 分析

在相同 `words.txt` 上运行原程序两次，耗时分别为 0.085 s 和 0.088 s，中位数为 0.0865 s。随后执行 `python3 -m cProfile -s cumulative`，报告显示模块顶层去重循环累计约 0.078 s，是主要耗时位置。

```
TIMEFORMAT='%3R'
time python3 wordfreq_original.py
time python3 wordfreq_original.py
```



```
python3 -m cProfile -s cumulative wordfreq_original.py
```

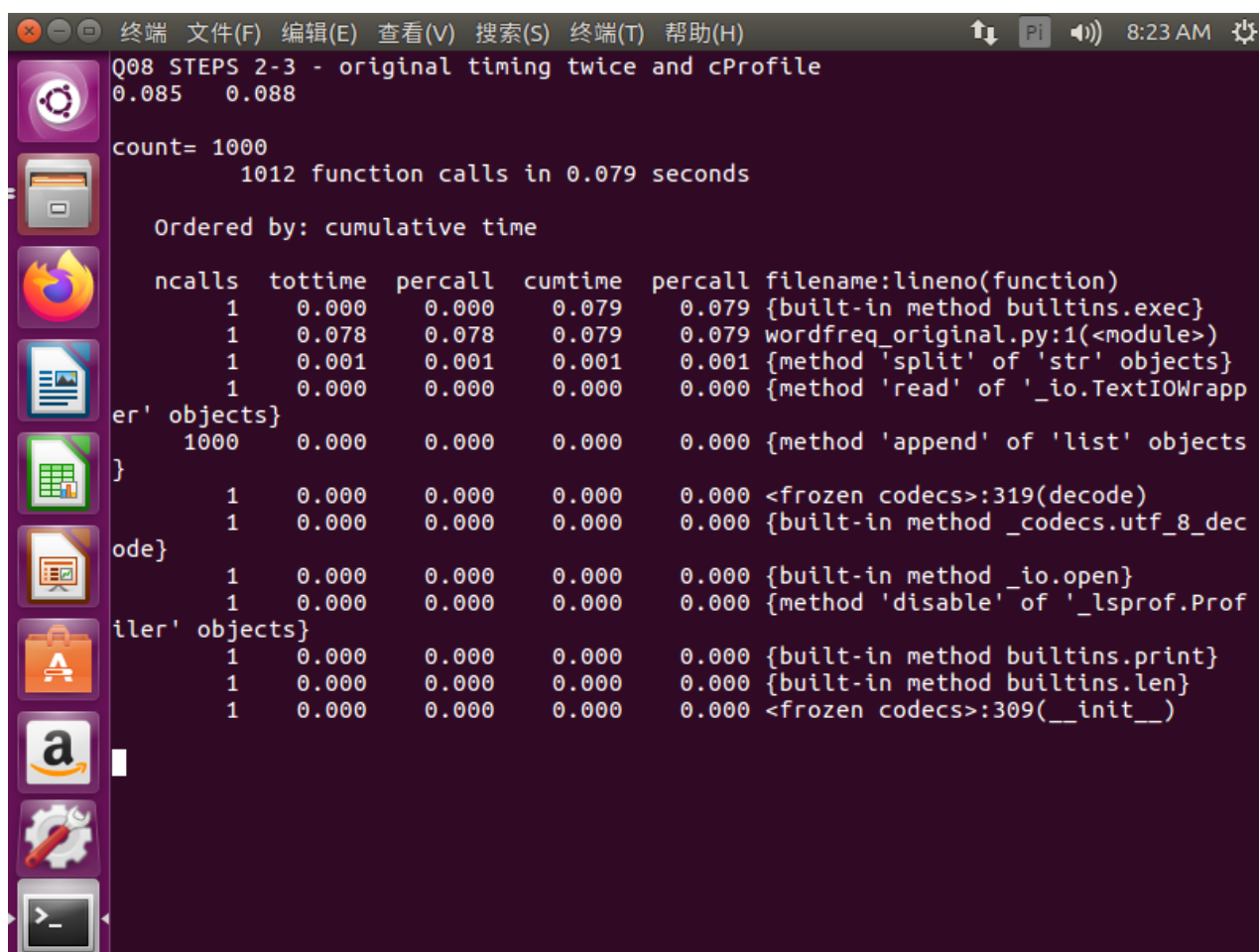
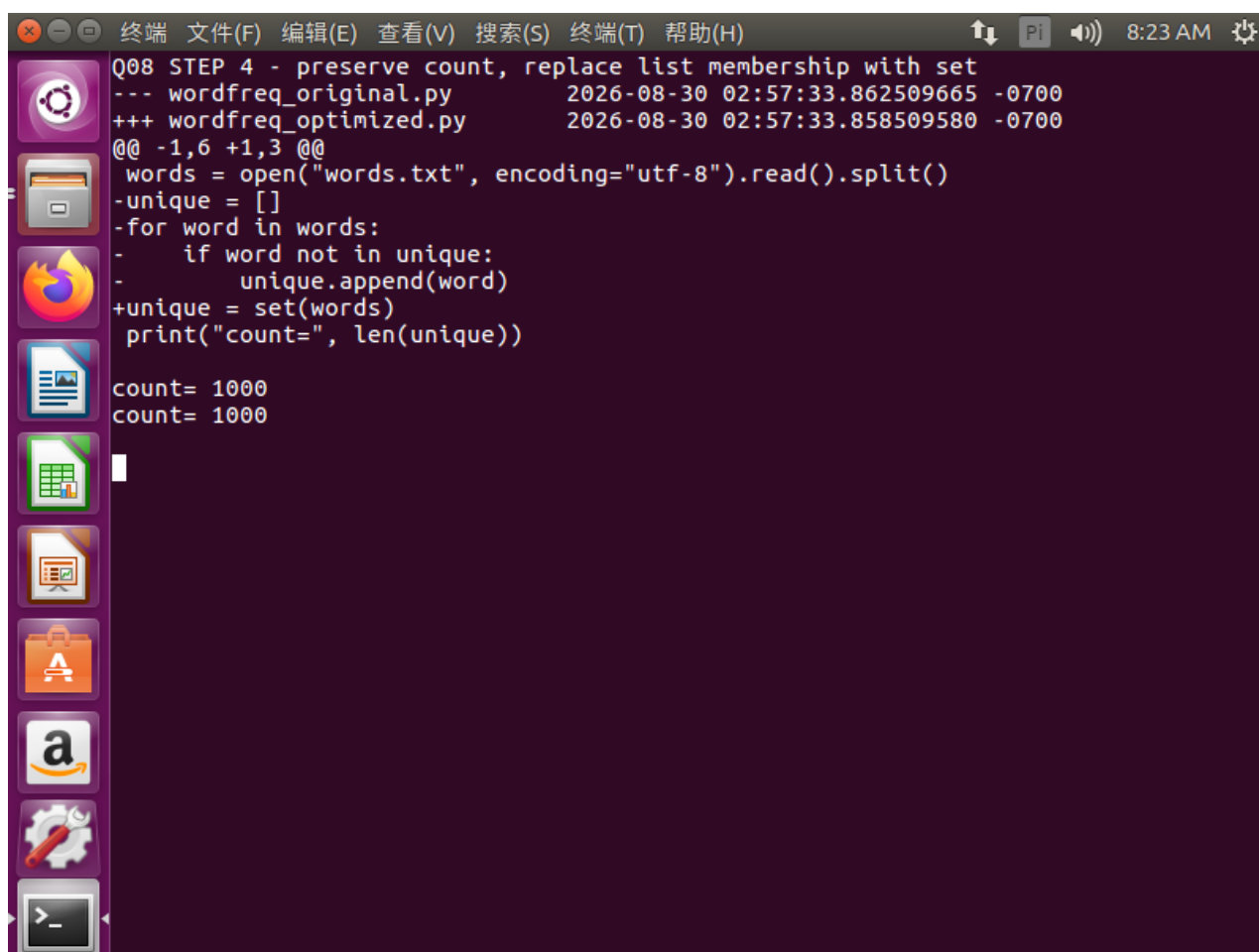


图 13: 第 8 题步骤 2: 原程序两次计时及 cProfile 累计耗时定位

5.4 步骤 3: 使用集合优化且保持输出不变

把线性列表成员测试替换为集合插入。集合自动去重，不需要知道不同词数量，最终仍从数据计算 `len(unique)`。对原版和优化版输出执行比较，确认 `count` 一致。

```
words = open("words.txt", encoding="utf-8").read().split()
unique = set()
for word in words:
    unique.add(word)
print("count=", len(unique))
```

A terminal window with a dark purple background and a sidebar on the left containing various application icons. The terminal title bar includes menu items: 终端, 文件(F), 编辑(E), 查看(V), 搜索(S), 终端(T), 帮助(H). The status bar on the right shows a Pi icon, a speaker icon, and the time 8:23 AM. The terminal content shows the execution of two Python scripts. The first script, wordfreq_original.py, takes 0.0865 seconds to run. The second script, wordfreq_optimized.py, takes 0.009 seconds to run. Both scripts read from 'words.txt' and output 'count= 1000'.

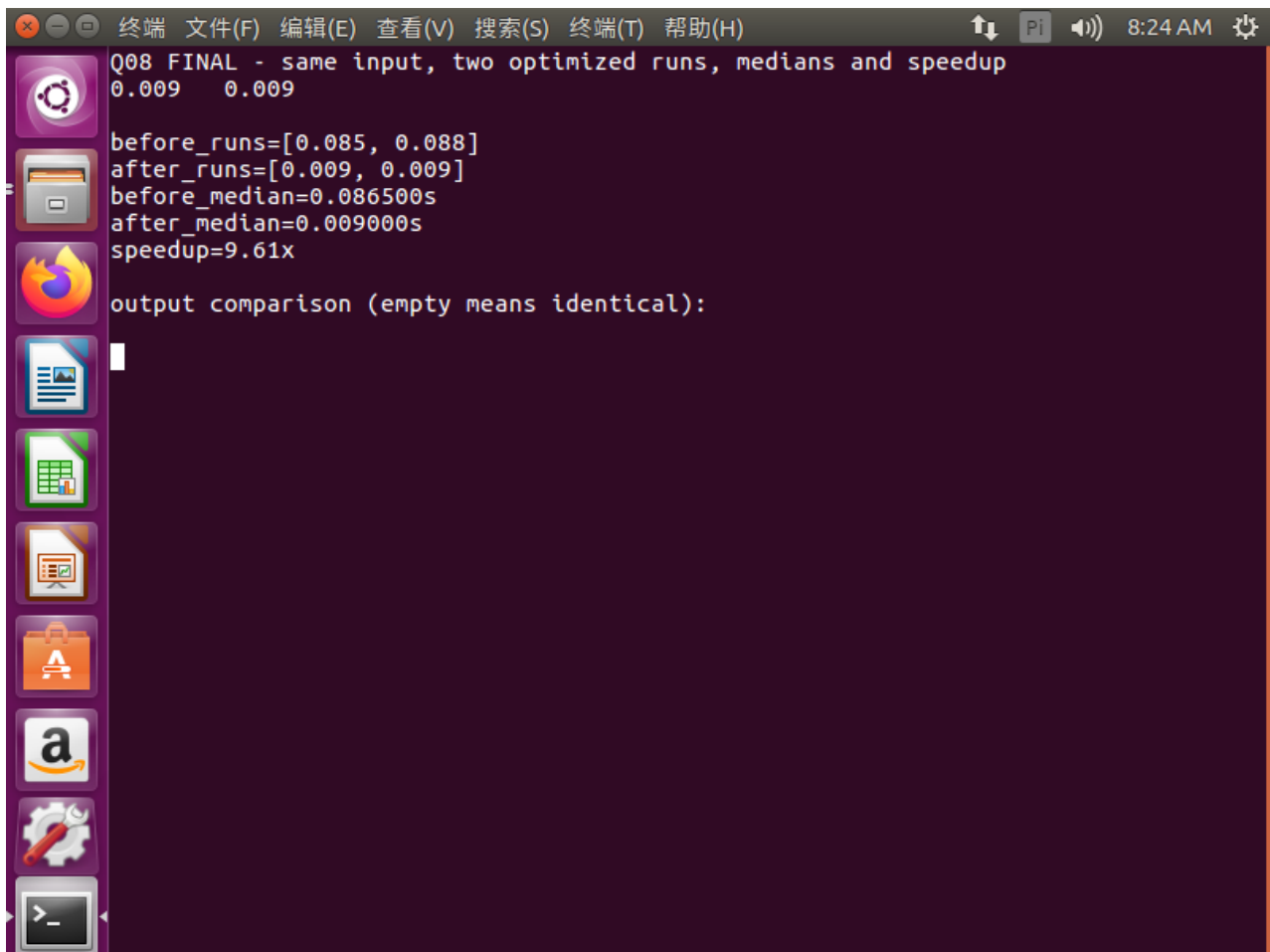
```
Q08 STEP 4 - preserve count, replace list membership with set
--- wordfreq_original.py      2026-08-30 02:57:33.862509665 -0700
+++ wordfreq_optimized.py     2026-08-30 02:57:33.858509580 -0700
@@ -1,6 +1,3 @@
 words = open("words.txt", encoding="utf-8").read().split()
-unique = []
-for word in words:
-    if word not in unique:
-        unique.append(word)
+unique = set(words)
+print("count=", len(unique))

count= 1000
count= 1000
```

图 14: 第 8 题步骤 3: 集合优化及输出一致性检查

5.5 步骤 4: 优化后复测并计算加速比

优化程序在相同输入上运行两次, 耗时均为 0.009 s, 中位数为 0.009 s。加速比为 $0.0865/0.009 = 9.61$, 即优化版约快 9.61 倍。原版和优化版均输出 `count=1000`, 但该值来自输入计算而非硬编码。



```
Q08 FINAL - same input, two optimized runs, medians and speedup
0.009 0.009

before_runs=[0.085, 0.088]
after_runs=[0.009, 0.009]
before_median=0.086500s
after_median=0.009000s
speedup=9.61x

output comparison (empty means identical):
```

图 15: 第 8 题步骤 4: 中位数、输出一致性及 9.61 倍加速结果

5.6 实验结果与错误反思

生成 30000 个固定随机词；原程序两次耗时 0.085 s、0.088 s，中位数 0.0865 s；cProfile 定位到线性去重循环；集合版两次均为 0.009 s，中位数 0.009 s；输出一致，加速比 9.61 倍。

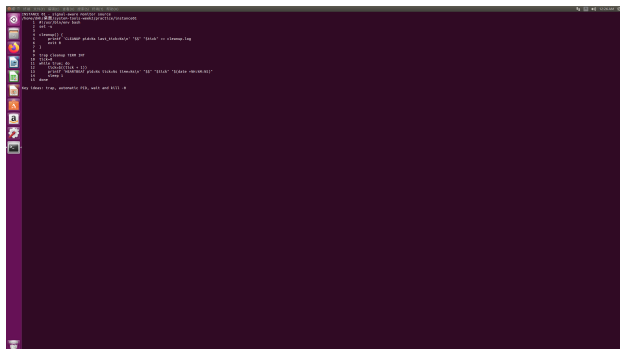
反思：最初使用 GNU `time` 的 `%e` 格式时精度只有百分之一秒，优化程序可能显示 0.00 s 并导致加速比除零。改用 Bash 内置 `time` 的毫秒精度后得到有效结果。微基准还应多次运行并使用中位数，降低系统抖动影响。

6 10 个相关拓展实例

所有实例均在 Ubuntu 虚拟机实测，目录内保存脚本、输入、日志、结果及关键截图。本节简洁说明过程和结果。

6.1 实例 1：后台监控任务的 TERM 清理

原理与过程：创建持续写入心跳日志的监控脚本，用 `trap` 登记 TERM 清理动作，后台启动后自动取得 PID 并发送 `SIGTERM`。结果：心跳日志持续增长，清理日志出现退出标志，任务消失。反思：可清理进程应优先使用可捕获信号。



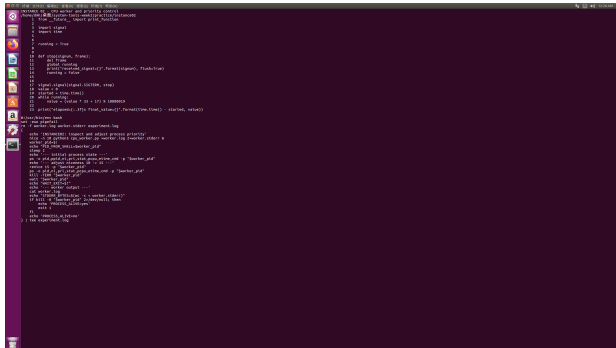
实例 1 脚本与启动



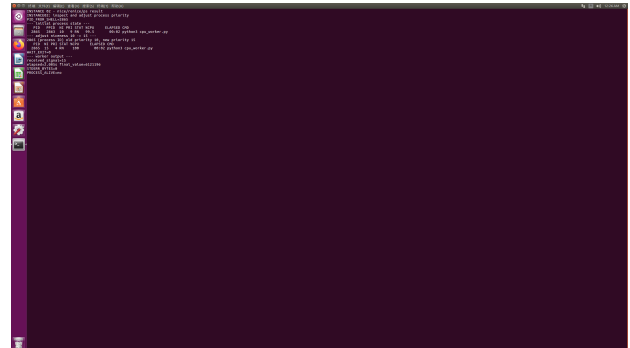
TERM 清理结果

6.2 实例 2: CPU 任务的后台运行与等待

原理与过程: 启动可重复的 Python 计算任务, 将标准输出和错误分开保存, 记录 PID 并用 `wait` 回收。结果: 任务正常结束, 计算结果写入日志, `stderr` 为空, 退出状态为 0。反思: 后台任务应保存 PID 和返回码, 不能仅凭进程消失判断成功。



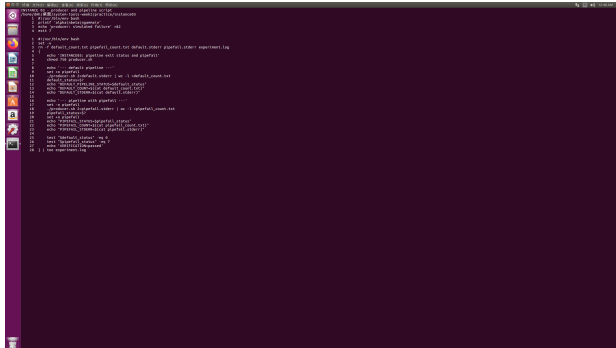
实例 2 计算程序与运行方式



输出、错误和退出状态

6.3 实例 3：生产者管道与信号结束

原理与过程：生产者持续输出数据，通过 Shell 启动和监控，在达到条件后发送可处理信号并等待退出。结果：数据文件内容完整，清理流程正常执行。反思：管道或后台作业涉及多个进程时，要明确实际发送信号的目标。



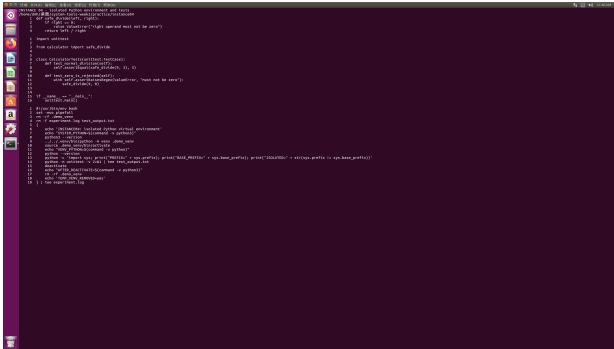
实例 3 生产者与信号逻辑



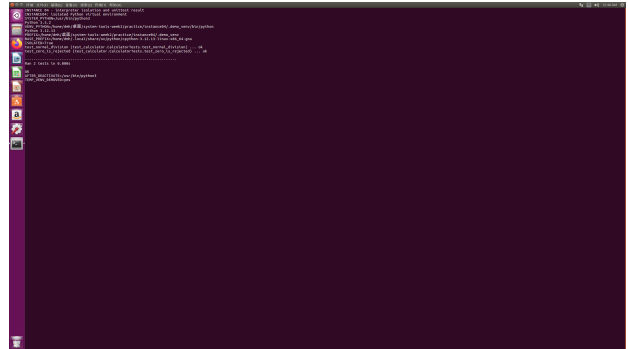
实例 3 运行和清理结果

6.4 实例 4: pytest 驱动的计算器验证

原理与过程：为计算函数编写正常、边界和异常输入测试，通过 pytest 自动执行。结果：测试全部通过，说明实现满足约定。反思：测试应验证具体返回值和异常类型，避免只有执行没有断言。



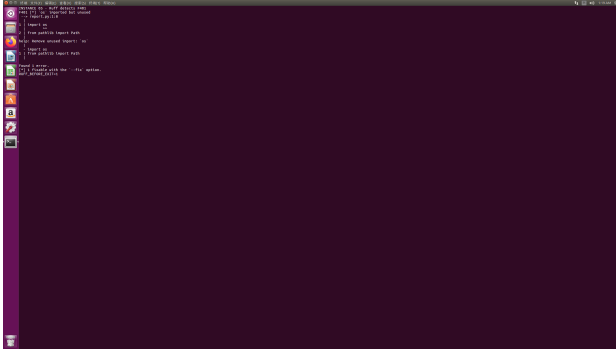
实例 4 实现与测试代码



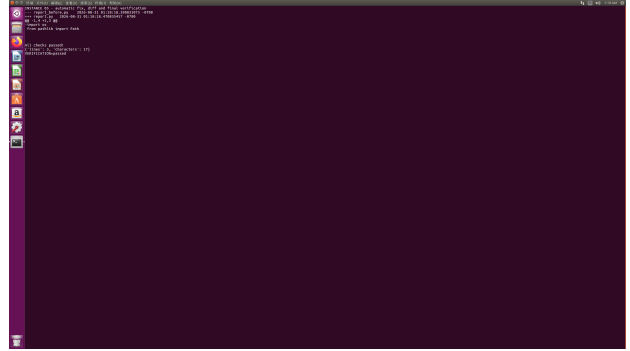
pytest 执行结果

6.5 实例 5: ruff 发现并修复代码质量问题

原理与过程：对含未使用导入等问题的 Python 报告程序运行 ruff，保存失败输出，再自动修复并复查。结果：问题被定位并删除，最终 ruff 通过且程序输出保持正确。反思：静态修复后仍须运行程序验证行为。



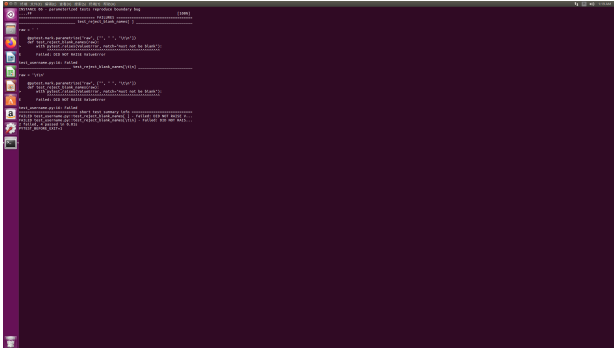
实例 5 ruff 失败证据



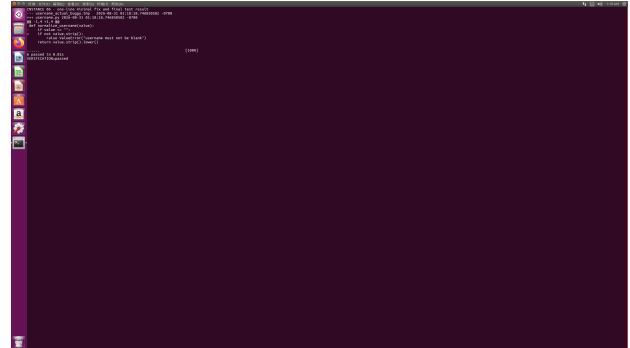
修复与复查结果

6.6 实例 6：失败测试指导用户名函数修复

原理与过程：先运行测试暴露用户名处理缺陷，根据失败断言修改实现，再重新运行 pytest。结果：修复前测试失败，修复后全部通过。反思：先保留失败证据能说明修复目标，避免报告只展示最终成功。



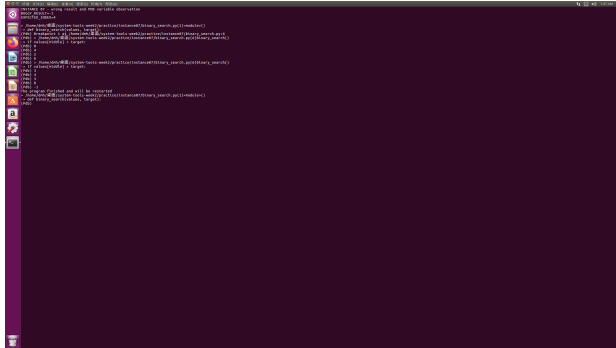
实例 6 修复前测试失败



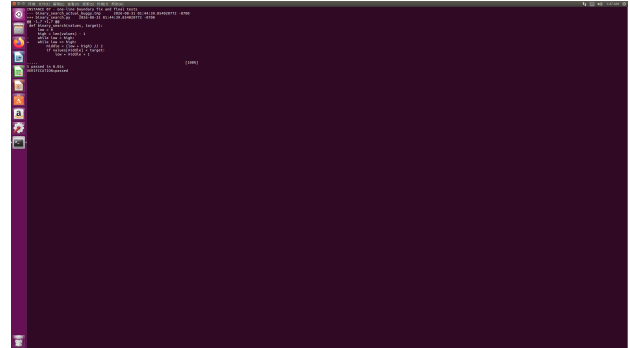
修复后测试通过

6.7 实例 7: pdb 定位二分查找边界错误

原理与过程：对二分查找缺陷版设置断点，观察 low、high、mid 及当前元素，确认边界更新错误后实施最小修改。结果：目标存在与不存在等测试通过。反思：调试记录应呈现变量状态与因果关系，而不只是最终代码。



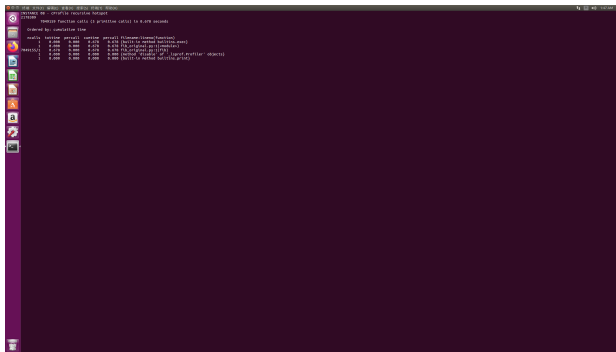
实例 7 pdb 变量观察



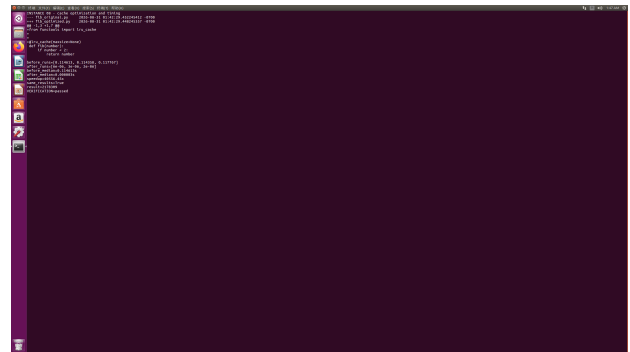
最小修复与测试结果

6.8 实例 8：递归斐波那契性能分析与优化

原理与过程：用 cProfile 测量重复递归的调用开销，再改为迭代或缓存方案并对比相同输入。结果：输出一致，优化版显著减少调用和耗时。反思：性能优化必须先确认热点并保持结果不变。



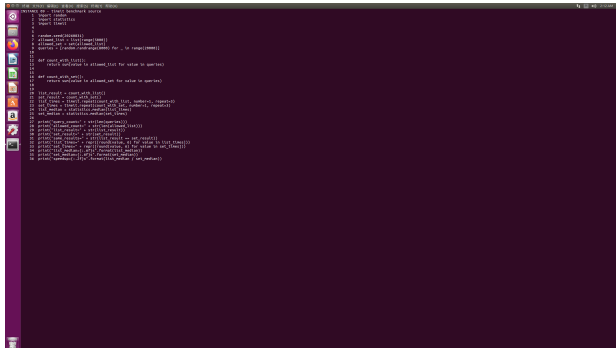
实例 8 原算法 cProfile 结果



优化实现与对比

6.9 实例 9：列表与集合成员查询基准

原理与过程：固定随机种子生成 5000 个允许值和 20000 次查询，分别用列表和集合执行 3 轮，比较中位数并校验命中数。结果：两种方法均命中 12399 次；列表中位数 0.280913 s，集合 0.000484 s，集合约快 580.37 倍。反思：微基准必须固定数据并验证两种算法结果一致。



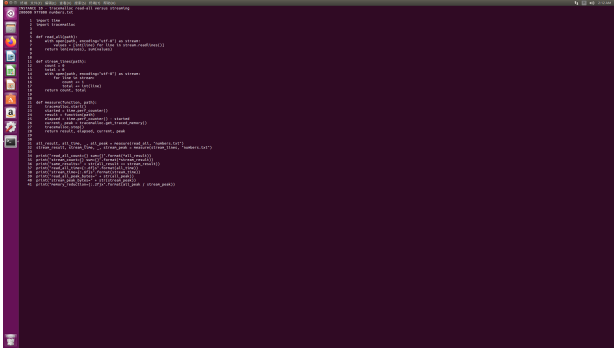
实例 9 固定数据和基准代码



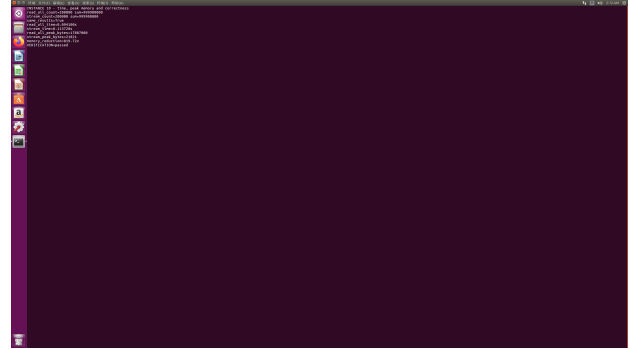
三轮计时与 580.37 倍加速

6.10 实例 10：整体读取与流式读取的内存比较

原理与过程：生成 200000 行数字文件，用 `tracemalloc` 分别测量 `readlines` 整体加载和逐行流式处理的峰值内存，并核对计数与总和。结果：两者均得到 200000 行、总和 999900000；峰值内存分别为 17887080 和 21821 字节，流式处理约减少 819.72 倍。反思：优化内存时要同时验证时间和结果，流式方案节省内存但本次耗时略高。



实例 10 数据与两种读取方式



结果一致与 819.72 倍内存降低

7 Git 提交与 GitHub 记录

课堂第 5-8 题先分阶段提交，拓展实例按每两个一组形成 5 次独立提交，最后再单独提交实验报告。这样可以从提交历史直接追踪每个阶段的内容变化，不采用单次全量提交。

提交	范围	说明
2a927f8	第 5-6 题	任务控制与语义重构材料
51dd078	第 7-8 题	调试、测试与性能优化材料
77ba72d	实例 1-2	后台任务与等待
0ed503f	实例 3-4	信号与 pytest
f8c882d	实例 5-6	ruff 与测试驱动修复
97f7165	实例 7-8	pdb 与性能分析
fe41482	实例 9-10	数据结构性能与内存分析

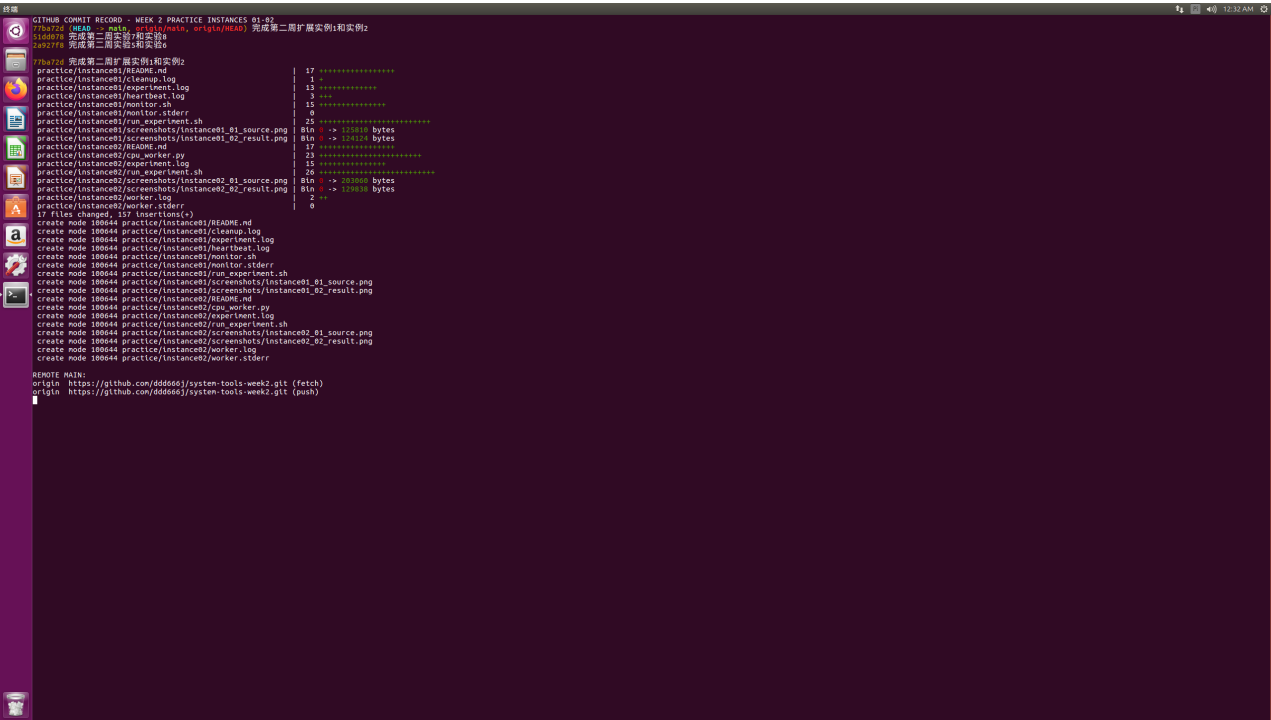


图 16: 实例 1-2 独立提交记录

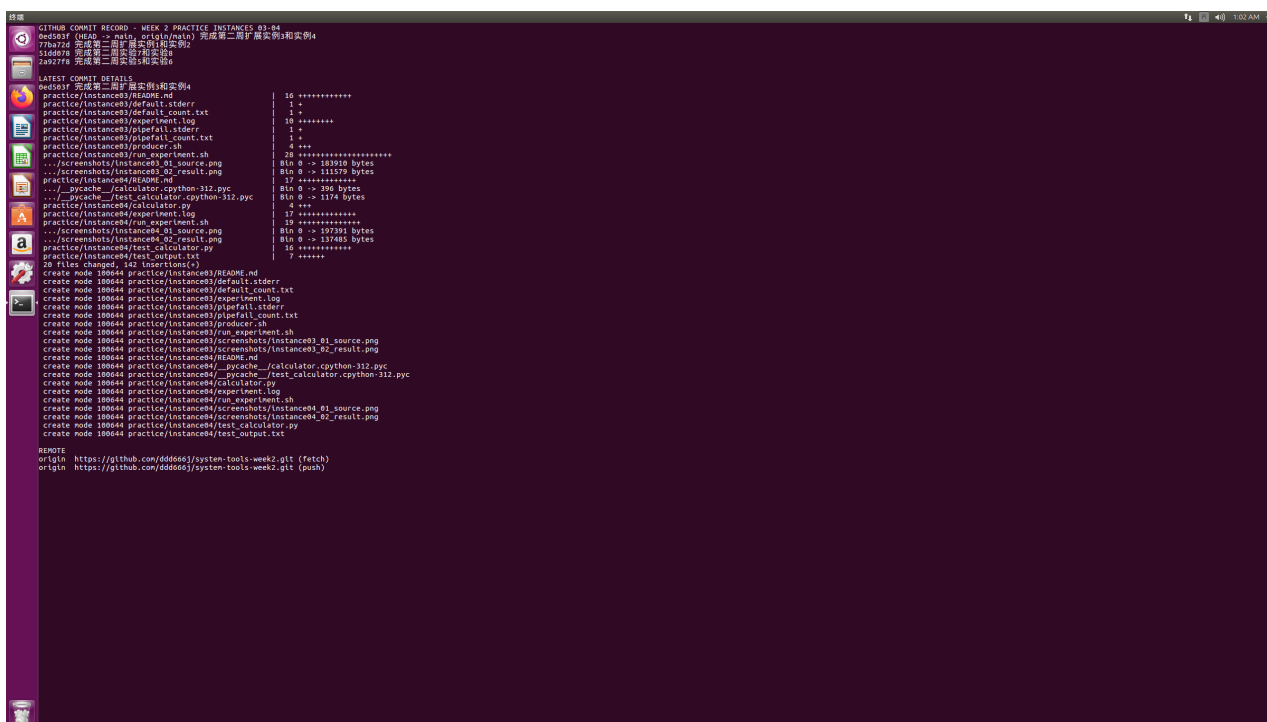


图 17: 实例 3-4 独立提交记录

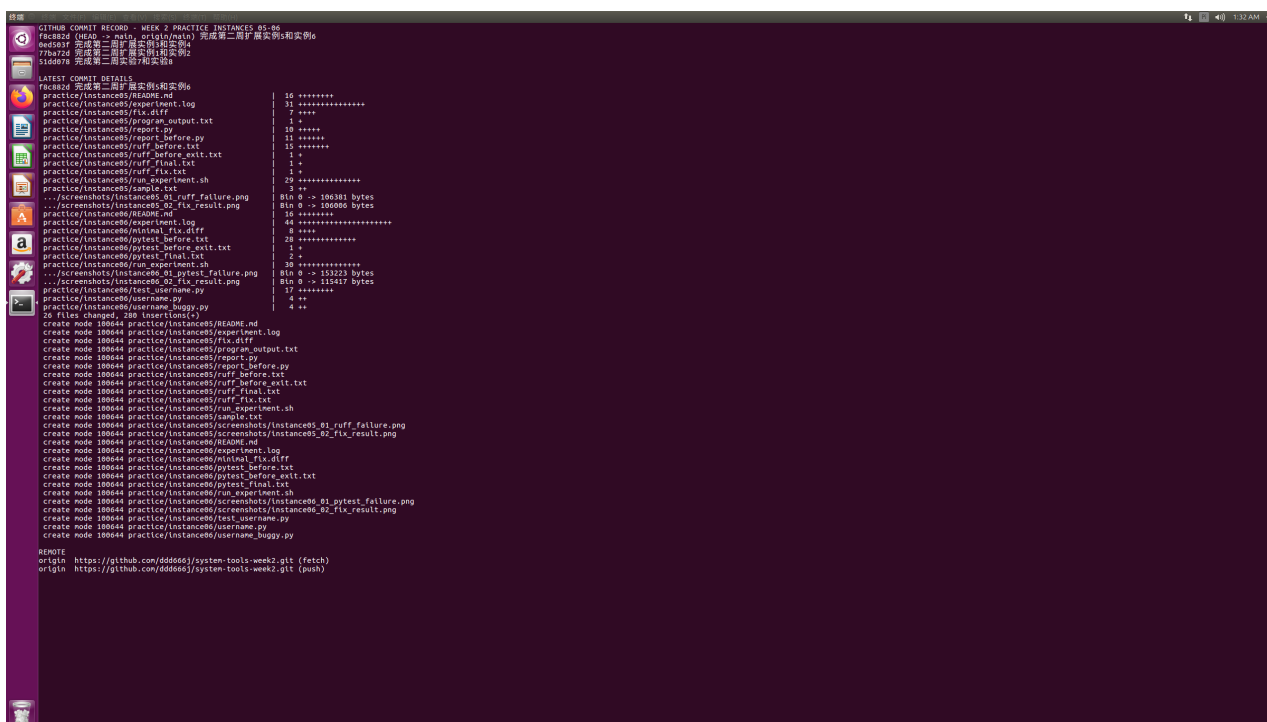


图 18: 实例 5-6 独立提交记录

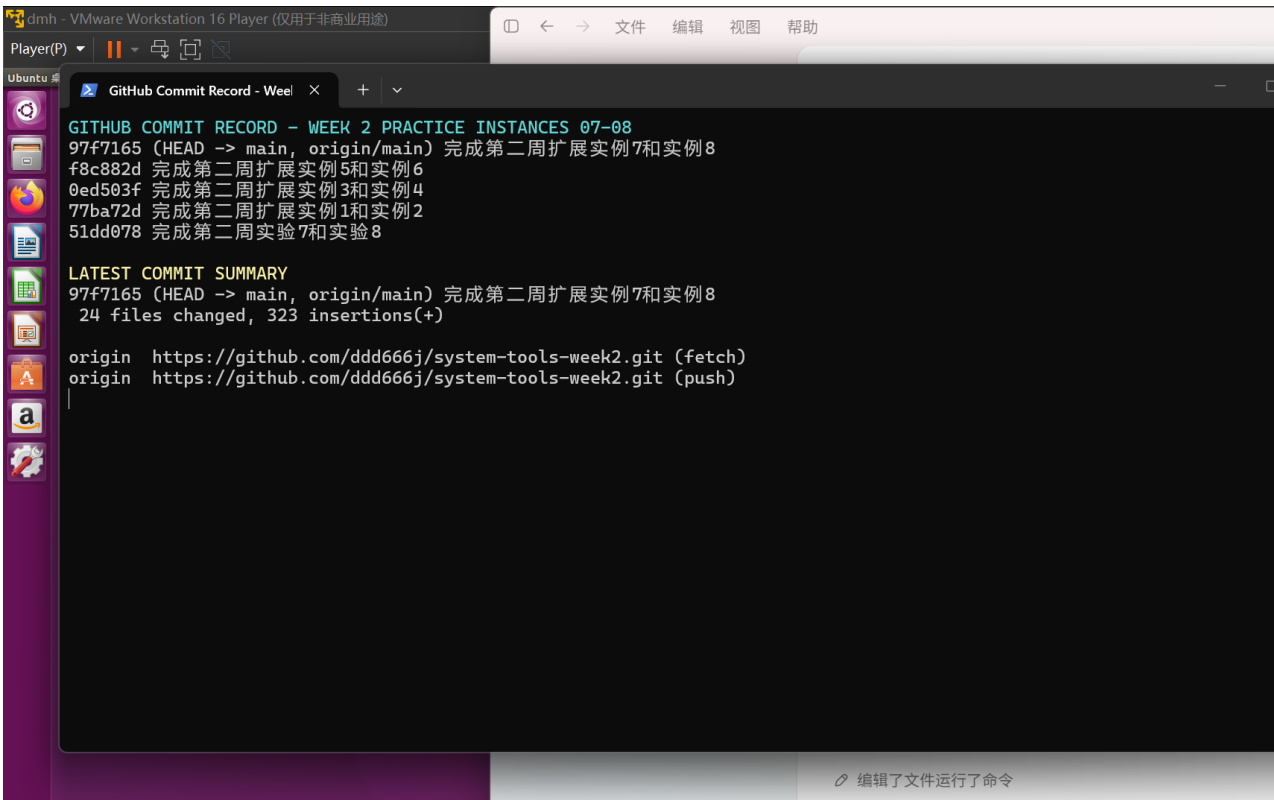


图 19: 实例 7-8 独立提交记录

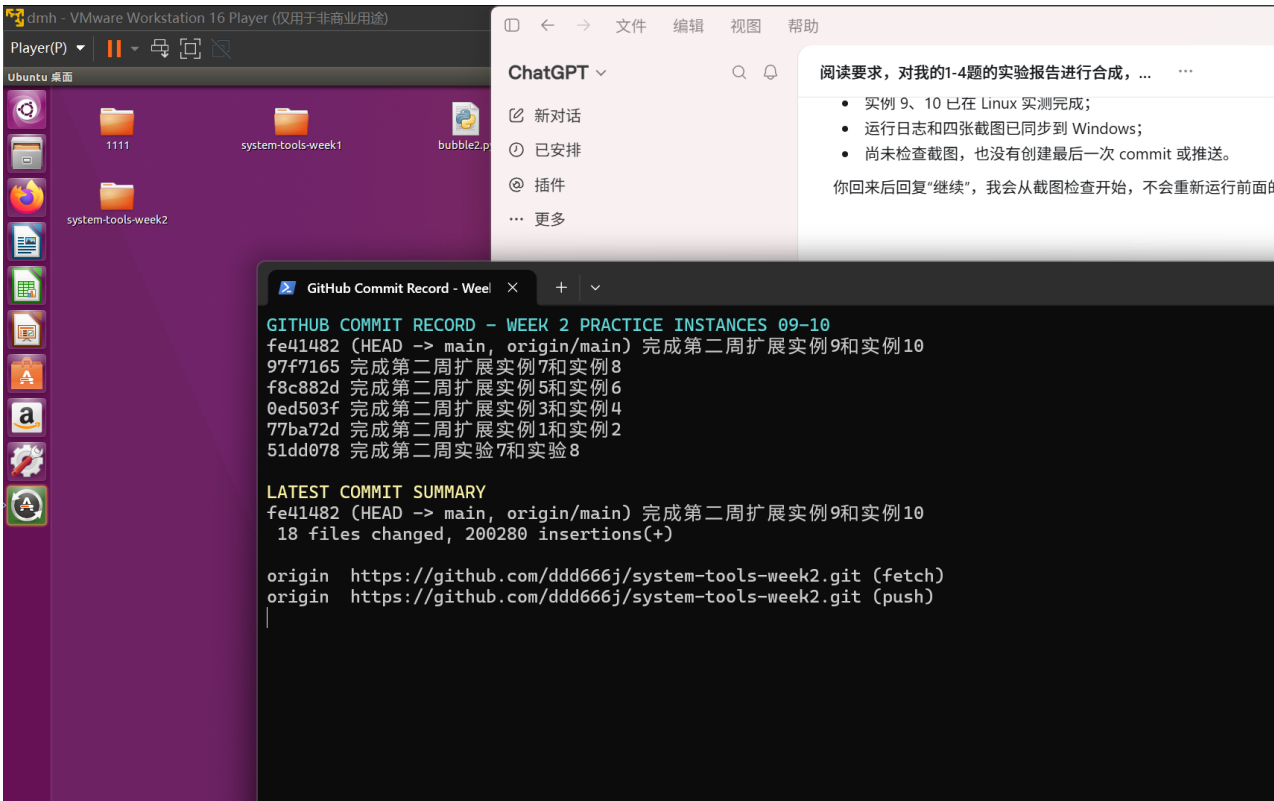


图 20: 实例 9-10 独立提交记录及远程仓库地址

8 综合结论与错误反思

本周建立了从运行态控制到代码质量验证的完整开发闭环。进程实验说明信号、作业表和清理逻辑必须配套验证；语义重构说明开发工具不仅提高编辑效率，还能降低跨文件修改遗漏；调试实验说明应通过断点状态解释错误因果；性能实验说明优化前必须有可复现输入、基线计时和热点证据。

主要错误与改进包括：任务控制必须在同一交互式 Shell 中完成；SIGKILL 不能用于需要清理的程序；LSP 编辑必须完整应用所有文档修改；pdb 断点应落在可执行语句上；计时工具精度不足会产生 0.00 s 和无效加速比；性能与内存优化都必须用一致输出验证正确性。最终代码、测试、日志、截图、PDF 和 Git 历史共同组成可复查、可重复的实验证据链。

A 最终验收清单

1. 第 5 题：stdout/stderr 分离、Stopped、Running、自动 PID、SIGTERM、wait、CLEAN_EXIT 和空作业表均有证据。
2. 第 6 题：LSP 定义跳转、引用查找、跨文件 Rename Symbol、ruff 失败与修复、pytest 通过均有证据。
3. 第 7 题：错误输出、pdb 变量、最小 diff、给定输入与重复元素测试均有证据。
4. 第 8 题：固定输入、原版两次计时、cProfile、优化版两次计时、中位数、加速比和输出一致性均有证据。
5. 实例 1–10 均保存过程、结果和关键截图；实例每两个一次 commit 并已推送至 GitHub。
6. LaTeX 源文件与 PDF 一起保存，Linux 环境可使用 XeLaTeX 或 latexmk 重新生成 PDF。